Stealth: The Next Chapter

Private Payments, Final Settlement and Quantum Security

James Stroud
Stealth R&D LLC
james@stealth.org

Sascha Pahlke
Stealth R&D LLC
sascha@stealth.org

Abstract

Stealth is a proof-of-stake cryptocurrency whose ordinary transactions need no fee. This paper describes the new Stealth protocol, run by a node written in Rust, and the two problems it answers. A public ledger exposes every payment: anyone can read who paid whom and how much, for as long as the ledger exists. And a large quantum computer breaks the signatures that protect coins and the encryption that protects private data, while encrypted data recorded today can be read once such a computer exists.

Stealth answers the first problem with a second chain beside the transparent one: a shielded sidechain in which coins are notes that appear on the ledger only as commitments, are spent by publishing markers that cannot be linked to them, and move under zero-knowledge proofs that show a payment to be valid without revealing its sender, its recipient or its amount. It answers the second by building the shielded chain from hash-based proofs and post-quantum encryption, by signing blocks and finality votes with post-quantum signatures, and by letting holders of transparent coins protect them with a spend that commits to its destination before it reveals its key. A committee of stakers makes blocks final, and the chain itself holds the coins that cross between the two chains, releasing them only against proof that their shielded counterpart was destroyed. What follows is a payment system in which a payment can be private, final and free of fees, and in which value held privately stays confidential and unforgeable against an adversary with a quantum computer. The same foundations carry a private messenger, a light client that trusts no server and per-request payments by software agents.

Contents

I Introduction 3

1 Introduction 3

II The Node 7

2 The Rust node 7

3 One thread, one clock 9

III Privacy and the Sidechain 11

4 Privacy: the XSS shielded sidechain 11

5 The sidechain architecture 16

IV Security 20

6 Post-quantum security 20

7 BFT finality 22

8 Consensus custody and the XST ↔ XSS bridge 24

V Applications 28

9 Private messenger 28

10 Commit-reveal ("enclave") addresses 33

11 The trustless light client 35

12 StealthPay and x402 agentic payments 36

VI The Chain 40

13 Pruning 40

14 Feework hardening and block space 40

15 Microtx: the minimum output 43

16 The +100 StealthNodes 43

VII Closing Remarks 45

17 Closing remarks 45

Disclaimer 45

Part I
Introduction

1 Introduction

Stealth is a cryptocurrency with the currency symbol XST. Its chain reaches consensus by qPoS (quantum proof-of-stake): a set of "stakers" produce blocks in turn, each in its own five-second slot. Its transactions are fee-optional: a sender may pay a fee, or send without one as a feeless transaction. Its nodes can run over Tor through a built-in Tor client, and by default a node connects to its peers directly. The qPoS paper describes that chain; this paper does not describe it again and covers only what is new.

The new Stealth protocol is run by the Stealth node written in Rust. Under the existing chain's rules the Rust node judges blocks exactly as the original node written in C++ does: it starts from the same genesis block, joins the same network and reaches the same verdict on every block, so the two nodes accept the same blocks, byte for byte. The Rust node has replayed the entire existing chain, from its genesis block to its tip, alongside the C++ node, and the two agree on every block. This paper is not a proposal: it describes the protocol that the Stealth node written in Rust runs. Every new consensus rule in this paper enters at a fork height, and several rules can share one fork height; the set of rules in force at a block is determined by that block's height alone. A block below a rule's fork height is judged exactly as before, except that the block just below the finality fork must have a hash built into the node.

1.1 A public ledger exposes every payment

A blockchain is a ledger that everyone can check, and to be checkable it has to be readable. When Alice pays Bob 25 XST on a transparent chain, the transaction names the coin Alice spends, the address at which Bob receives and the amount, and every node keeps that record for as long as the chain exists.

Addresses carry no names, but they do not need to. Once Bob knows which address paid him, he can follow Alice's coins backwards to where she received them and forwards to everyone she pays after him, and he can read the balance she has left. Anyone who has tied a single address to a person, whether an exchange, a shop or the reader of a published donation address, holds the same view.

The linking is also mechanical. A transaction that spends several coins shows that one party held all of them, and the change that returns to the payer is often easy to tell apart from the payment. Bitcoin keeps names off the ledger by identifying owners with keys. A ledger of this kind is pseudonymous, not private, and its exposure is permanent: a payment made today can be analysed with any method devised afterwards.

1.2 Quantum computers break signatures and encryption

The signatures that protect coins on most chains, including ECDSA (the Elliptic Curve Digital Signature Algorithm) on Stealth's transparent chain, rest on one assumption: that the private key cannot be recovered from the public key. For elliptic curves that assumption is the hardness of the discrete-logarithm problem, and Shor's algorithm solves that problem efficiently on a sufficiently large quantum computer. The same holds for the elliptic-curve key exchange that many systems use to encrypt data to a recipient. Hash functions fare differently: the best known quantum attack on them, Grover's algorithm, reduces the work of a brute-force search only to its square root, so a hash function with a large enough output remains secure.

The threats differ in time. A forged signature can steal only coins whose keys are exposed once a quantum computer exists; there is nothing to harvest in advance, and a coin moved to safety before then is out of reach. Encryption is the opposite. An adversary can record ciphertexts today and decrypt them once the computer exists, so the exposure begins at the moment a ciphertext is transmitted. This is called "harvest now, decrypt later". A third threat concerns consensus itself: an adversary who can forge a block producer's signature can sign blocks in that producer's name, and so rewrite history or produce conflicting blocks.

For a transparent coin the exposure has a precise shape. A pay-to-public-key-hash coin, the standard kind, shows only a hash of its public key while it sits unspent. The key itself appears in the transaction that spends the coin, and it becomes public as soon as that transaction is relayed, before it is in any block. From that moment a quantum adversary has a window in which to derive the private key and race the honest spend with one of its own. A key that is already on the ledger, because its address was reused or has spent before, is exposed permanently.

1.3 How other systems approach these problems

In "CoinJoin", several users build one joint transaction, so that an observer cannot tell which input paid which output. This breaks some links, but amounts and participation remain visible, the joint transaction has a recognisable shape, and the users have to coordinate. The set of coins a payer hides among is the set of participants in that one transaction.

Monero hides the spent coin among decoys taken from the chain by means of ring signatures, gives every payment a fresh one-time address and hides amounts in commitments. Privacy is mandatory, so every user adds to every other user's cover. The cost is that each spend hides among its ring, a small set of decoys, rather than among all coins, and the choice of decoys is itself open to statistical analysis.

Zcash uses the note-based design on which this paper builds. Value is held as "notes" that appear on the ledger only as commitments; spending a note publishes a "nullifier" that marks it spent without saying which note it was; and a zero-knowledge proof shows that the transfer is valid. A spend hides among every note in the shielded pool. Shielding in Zcash is optional, so much of its activity is transparent and the amounts that move in and out of the pool are public. Its earlier proof systems required a trusted setup ceremony; its newest one does not. Bitcoin, Monero and Zcash rest their signatures, and where they encrypt, their encryption, on elliptic curves, which a large quantum computer breaks.

The direct answer to the quantum threat is to replace every signature with a post-quantum one. The cost is size: a standardised lattice-based signature is dozens of times larger than an ECDSA signature, so each payment takes correspondingly more block space. The most compact scheme selected for standardisation, Falcon, relies on sampling that is hard to implement without leaking the key through side channels. A chain can also require holders to move their coins to post-quantum addresses by a deadline, which leaves the coins of holders who miss it either exposed or frozen.

1.4 Stealth's approach

Privacy in Stealth is provided by a shielded sidechain, StealthSend (XSS). It follows the note-based design: notes appear on the ledger only as commitments, spends publish nullifiers, and every transfer carries a zero-knowledge proof. It differs in four ways. The proofs are STARKs (scalable transparent arguments of knowledge), which are built from hash functions alone and so need no trusted setup. Spend authority is knowledge of a secret whose hash the address publishes, not a signature. Notes are encrypted to their recipients with ML-KEM (the Module-Lattice-Based Key-Encapsulation Mechanism), a post-quantum scheme, so ciphertexts harvested from the ledger cannot be read later with a quantum computer. And shielded transfers carry no fee.

The transparent chain is protected where a quantum adversary would strike. Every block carries two signatures from its producer, an ECDSA signature and an ML-DSA-65 signature (ML-DSA is the Module-Lattice-Based Digital Signature Algorithm), and consensus requires both. The producers of private blocks sign with ML-DSA-65 as well. The post-quantum signature is made once per block, not once per transaction, so it adds nothing to the cost of a payment. Ordinary transparent spends keep their ECDSA signatures, and no coin is frozen or forced to move.

A holder who wants a transparent coin protected against a quantum adversary moves it to an "enclave" address. To spend from it, the holder first publishes a commitment to the key, the coin and the destinations, while the key is still secret; the ordinary spend that follows is valid only if a matching commitment is

in a final block. An adversary who derives the key from the spend can forge a signature but cannot redirect the coins, because their destinations were fixed before the key was known. A coin whose key is already on the ledger cannot be protected in place; it needs one sweep to a fresh address while ECDSA is still safe. A holder who wants privacy as well as protection uses the shielded chain instead.

Both protections rest on finality. In qPoS, as on most chains, settlement is probabilistic: a block becomes harder to reverse as blocks accumulate on top of it, but no depth is a guarantee. Stealth adds BFT (Byzantine fault tolerant) finality. The enabled stakers form a "finality committee", each member weighted by its qPoS weight, and each member casts a vote for the blocks it connects. A block is "final" when the members who voted for it hold more than two thirds of the committee's weight. Votes are signed with ML-DSA-65, and a block's finality certificate is the list of those signatures. A node never reorganises past a final block. As long as less than one third of the committee's weight is Byzantine, a final block is never reverted: two conflicting blocks could both become final only if members holding more than a third of the weight voted for both, and such double voting is provable and slashable.

Finality also lets the chain itself act as custodian for the coins that move between its two chains. No committee holds keys to them and no one signs their release; the chain releases them only against proof that the matching shielded value was burned, recorded under a root that the finality committee has ratified. The one assumption beneath all of this, beyond the cryptography, is the one already placed in every block: that less than one third of the committee's weight is Byzantine.

1.5 Two chains

A shielded pool could be built into the transparent chain itself. Stealth instead places it in a second chain, the "private chain", which runs beside the "main chain" inside the same node. The two kinds of state behave differently. The main chain is the record that every transparent coin descends from, and it is never pruned. The private chain carries the proofs of shielded transfers, which are no longer needed once their effects are in every node's state, and by default a node deletes private blocks older than a recent window and keeps its full shielded state. A failure in the shielded machinery is contained: the main chain keeps running.

The two chains move in step (Figure 1). Each private block is bound by hash to one main-chain block: from the private chain's first block on, every main-chain block has one private block bound to it, and the private chain counts its own heights. The staker that produced the main-chain block produces the private block, and the finality committee decides, for each private height, either that producer's block or a "fallback block" that every node builds identically from the main chain alone. Every node applies only the decided block. The private block names its predecessor, commits to the shielded actions it contains, and commits to the roots of the shielded state that results. One thread in the node processes every consensus event in order: when a main-chain block is connected, the same thread advances the private chain bound to it. The node therefore never holds a private chain that disagrees with the main chain beneath it.

Value crosses between the chains through a bridge in which the chain itself is the custodian. Suppose Alice holds 100 XST and shields 60. She makes two deposits of 30 XST on the main chain, each naming the commitment of a new note; the chain locks them in outputs that only its release rule can spend, and once the main-chain block is final, two notes worth 30 XSS each are minted to her in the private chain. She pays Bob 25 XSS privately, spending both notes, and keeps a note of 35 XSS as change. Bob then unshields his note: he burns it, naming the address at which he wants to be paid, and the chain releases 25 XST from the lock to that address once it holds proof that the burn was recorded in a state the finality committee ratified. Throughout, the amount minted minus the amount burned equals the amount locked: $60 - 25 = 35$ XST remain locked, backing exactly the 35 XSS that remain shielded.

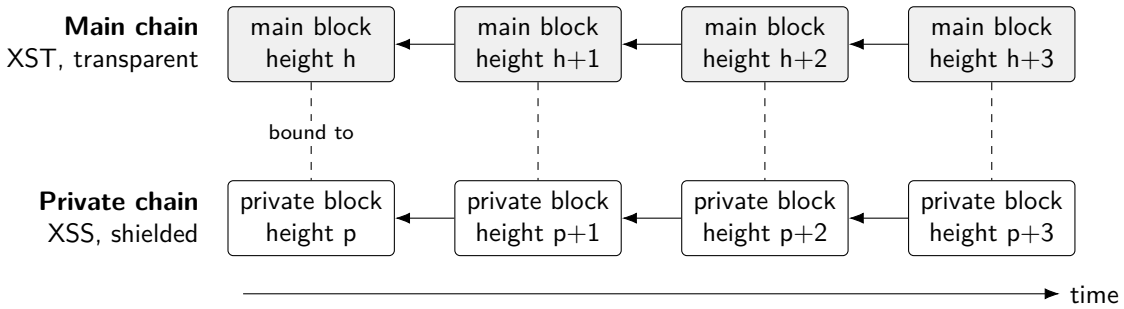


Figure 1: *The two chains. Each block names its predecessor (arrows). Each private block is bound by hash to one main-chain block (dashed), and the private chain counts its own heights. A private block is either the block of the staker that produced that main-chain block or the fallback block that every node builds, as the finality committee decides.*

1.6 Organisation of this paper

Part II describes the Rust node and its single consensus thread, which runs both chains with one clock. Part III treats privacy: Section 4 describes the shielded sidechain, namely notes, commitments, nullifiers, the spend proof, keys and note encryption, and states what an observer can and cannot learn; the section after it describes how the private chain is produced, bound to the main chain, validated and synchronised. Part IV treats security: post-quantum security, BFT finality, and consensus custody with the bridge between XST and XSS. Part V describes applications built on these foundations: the private messenger, commit-reveal enclave addresses, the trustless light client for desktop and mobile, and StealthPay with x402 payments by software agents. Part VI treats the chain itself: pruning, feework hardening and block space, the lower minimum output that allows microtransactions, and the +100 StealthNodes that expand the staker set. Part VII shows how the parts compose into the properties stated above.

Part II
The Node

2 The Rust node

A cryptocurrency's rules exist in the behaviour of its nodes. Each node checks every block for itself and follows the chain it finds valid, so two nodes that disagree about a single block follow different chains from that block on. A second implementation of a running chain therefore has to reproduce the first exactly: for every block in the history, and for every rule that history has seen, it has to reach the same verdict and produce the same bytes.

The Stealth node written in Rust meets that requirement. Under the existing chain's rules it accepts exactly the blocks, byte for byte, that the original node written in C++ accepts, and rejects the rest. It is also the single engine for every client: one node engine is the canonical engine, and the private chain runs inside the same node process, with no separate program for it. This section describes how the Rust node stays on the one chain, how it keeps every computation reproducible, how it adds the new rules of this paper without disturbing the old ones, and what it assumes.

2.1 One chain, two implementations

The Rust node and the C++ node begin at the same place and speak the same language. The Rust node starts from the same mainnet genesis block, the first block from which every other block descends. It uses the C++ node's peer-to-peer "network magic", the fixed bytes with which every message on the Stealth network begins, so both nodes run on one network and exchange blocks with each other. And it applies each historical rule change of the existing chain at the same height as the C++ node, so that a block from any period of the chain's history is judged by the rules that were in force at its height.

The node also computes the chain's own values in the same way. It forms the producer order of each qPoS round with the same seeded Mersenne Twister shuffle as the C++ node. A Mersenne Twister is a pseudo-random number generator: from a starting value, the "seed", it produces a long sequence of numbers that look random but are fully determined by the seed, so every node that starts from the same seed, drawn from the chain as the qPoS paper describes, shuffles the stakers into the same order. The Rust node computes each staker's qPoS weight by the same formula as the C++ node. And it judges whether a feeless transaction's fework is sufficient by the same rule.

One hash function of the existing chain is not reimplemented: the Rust node computes X13, the block hash in every era of the chain, whose proof of work secured only the chain's first 5,460 blocks, from the same C source files that the C++ node compiles. Each node writes only its own wrapper around those sources, the chaining of the thirteen hashes, so for X13 itself the question of whether two implementations agree does not arise. The node computes the Argon2d fework hash in Rust, and that implementation is checked against test vectors of the Argon2d reference implementation on the fework's settings.

2.2 Every node computes the same value

Agreement between nodes requires more than the same rules. It requires that each rule yields the same number on every machine. Amounts in the node are 64-bit integers, counted in a fixed smallest unit of XST, so an amount is never a fraction that different machines could round differently.

Square roots show why this matters. Computed in floating point, a square root is an approximation, and the last bits of an approximation can depend on the processor and the compiler. A weight that differed by one on one node would change that node's finality tally and the committee root it computes, and the node would reject a boundary block that every other node accepts. Square roots in consensus are therefore computed with an integer Newton iteration: starting from a guess x above the root of n, each step replaces x with the integer part of $(x + n/x)/2$, and the iteration stops when the value no longer falls. For $n = 1000$ and a first guess of 32, one step gives $\lfloor (32 + 31)/2 \rfloor = 31$, and the next gives 31 again, so the result is 31; indeed $31^2 = 961 \leq 1000 < 1024 = 32^2$. Every step

is integer arithmetic, so every node computes the same value.

2.3 New rules at fork heights

A running chain cannot change its rules for blocks it has already accepted, and every node has to switch to a new rule at the same block. Stealth solves both problems with fork heights. Every new consensus rule enters at a fork height, below which it is dormant, and several rules can share one fork height. The rules in force at a block are a pure function of that block's height, namely the rules of the last fork whose height the block has reached, and validation applies a new rule only to blocks at or above that rule's fork.

An example makes this concrete. Suppose rule A has fork height 100 and rule B fork height 200. A block at height 99 is judged by the existing rules, a block at height 150 by the existing rules and A, and a block at height 250 by the existing rules, A and B. No node needs to agree with any other on when to switch; each reads the height of the block in front of it. A block below every new fork is judged by the existing rules, and the block just below the finality fork must also have a hash built into the node; under the existing rules the two nodes accept exactly the same blocks, and every new consensus rule in the rest of this paper enters the chain this way.

2.4 One process, one engine

The Rust node runs both chains. The private chain described in Section 4 runs inside the one node process; there is no separate program for it that could fall out of step with the main chain. And the same engine serves every client: the mobile apps for iOS and Android, Node.js applications (among them the x402 agent of StealthPay), the browser through WebAssembly, the command-line wallet, the node itself, and the payment services, the x402 facilitator and StealthPay. Every client of the protocol therefore relies on the same implementation of its rules, rather than on several implementations that might disagree.

2.5 Security and assumptions

The Rust node has the "checkpoints" of the C++ node, and from the finality fork on also newer ones of its own; the block just below the finality fork must also have a hash built into the node. A checkpoint is the hash of a known block at a known height, built into the node. A chain that does not contain the checkpointed block at that height is refused, so no rival history can replace the checkpointed past: before the finality fork, as in the C++ node, for the eras before qPoS, and from the finality fork on in every era. The newest of these checkpoints, not the hash below the finality fork, marks where a syncing node starts re-verifying every transaction signature, as the C++ node does; from the finality fork on, every block below it is fixed by the chain of hashes that leads to a checkpointed block, so its signatures need no second check.

Rust is a programming language whose compiler rejects programs that could use memory after releasing it or let two threads write the same memory at once, and which checks every array index against the array's length at run time, stopping the program instead of letting it read or write outside the array. It enforces these guarantees everywhere except in code marked "unsafe", which the programmer vouches for instead of the compiler. Outside the cryptographic component's bindings, the node's consensus code contains no unsafe code, so the consensus code itself runs under the compiler's full guarantees. The consensus rules call C libraries through bindings, among them X13 and secp256k1, the library that checks ECDSA signatures. Calling a library written in C is itself unsafe in Rust, because the compiler cannot check the other side, so the node confines its own unsafe code to the cryptographic component's bindings; there it lies in one place and can be examined.

The node's agreement with the C++ node rests on three assumptions: that the two nodes share the genesis block, the network magic, the historical fork heights and the C++ node's checkpoints; that the C sources both nodes use for X13 compute the same results when built into either node; and that the node's Argon2d gives the same output as the reference implementation the C++ node uses, which its test vectors check. Sharing the X13

sources is what makes agreement on that hash certain, and for the same reason a flaw in them would affect both nodes alike rather than show up as a disagreement between them.

2.6 From today's chain to the new protocol

The network moves to the protocol of this paper in three steps, and no coin has to move at any of them. In the rollover, C++ and Rust nodes run side by side on today's rules, with 1-megabyte blocks; the Rust node judges every block as the C++ node does, so both stay on one chain while users, stakers and services change to the Rust node. At the finality fork, the cutover, the main chain's new rules switch on together: BFT finality, post-quantum consensus signatures, blocks of 2 megabytes, microtransactions, the feework hardening and the newer checkpoints. Its height is set before the release that carries it, and a node that has not upgraded by then is left behind. The fork after the finality fork switches on the private chain with consensus custody and the bridge, commit-reveal enclave addresses, the additional 100 StealthNodes and the private messenger.

3 One thread, one clock

The Rust node runs two chains, the main chain and the private chain bound to it, and it must keep them in step. If each chain ran on its own thread with its own timer, each would advance at its own pace, and every exchange between them, a private block bound to its main-chain block or a deposit minted once it is final, would have to be reconciled across threads, with a result that could depend on which thread happened to run first. The node avoids this by giving both chains one clock and one thread.

3.1 The chain is the clock

Stealth's qPoS chain keeps time by its blocks. In each qPoS round every enabled staker holds one five-second slot, in the round's shuffled order, and a block is valid only from the staker whose slot is current, one block per slot. Each block is a tick of the network's clock: the schedule advances on the timestamps of the blocks themselves, not on any node's clock, and no node asks its peers for the time or averages their answers. A Rust node's own clock enters consensus at one point only: the node refuses a block dated later than its own present. A producer whose clock runs fast risks having its blocks refused, so every producer has reason to keep good time, and no node has to trust the clock of another.

This is deterministic timekeeping: every question of time that consensus asks is answered from the blocks themselves, by counting them or by reading their timestamps, so every node that holds the same blocks reaches the same answer, whatever its own clock shows. The durations below are counted in blocks, not in seconds. A feework expires 24 blocks after the block it names; the feework of a private action, once the main-chain height to which its private block is bound lies more than 24 blocks above the block it names; and a commitment of a commit-reveal address can be revealed once its block is final, until the tip has passed its inclusion height by 512 blocks plus a reveal window of 720 blocks, about an hour.

3.2 One thread

The node has a single consensus thread, and every consensus event is a command to it: connecting a block received from a peer, producing and connecting the block of the node's own staker slot, admitting a transaction from a peer or from the node's own interface, and merging the finality signatures and votes that peers send. The thread takes the commands one at a time, in the order in which they arrive. The network threads receive messages from peers and hand each block, transaction and vote to it, and the staker's thread hands it each slot to produce, so that the block is built against exactly the tip that it extends.

The private chain has no timer of its own. When a block connects or the node's own block is produced, and the tip moves, the same thread then advances the private chain, binding it up to the newest final main-chain block. When a private block arrives from its producer, the arrival only wakes the thread to take the same step sooner; it changes when the private chain advances, never what it adopts. The private chain's clock is therefore the main

chain's: it steps at main-chain blocks, on the same thread, in the same order on every node.

The zero-knowledge proofs of a received private block's actions depend on no shared state, so they are verified concurrently on worker threads; the verdicts come back, and the thread then applies the actions to the shielded state one after another, in their fixed order. The actions of the node's own block were each verified when they arrived.

3.3 What one clock gives

Each step sees the state that the step before it left, so the node never reconciles two views of its own state. The main chain and the private chain advance in one sequence, and when the main chain reorganises beneath the private chain, the next step either rewinds the private chain or halts it for a resynchronisation. Every node orders the same blocks the same way and counts the same durations in the same blocks, so all nodes agree on which transactions have expired, and a node whose clock is slow or fast changes none of it.

Part III
Privacy and the Sidechain

4 Privacy: the XSS shielded sidechain

On the transparent chain, a payment from Alice to Bob names the coin Alice spends, Bob's address and the amount. Every node needs these facts to check three things: that Alice owned the coin, that she had not spent it before, and that she did not create value. The shielded sidechain, XSS, lets every node check the same three things without learning any of the three facts. This section explains how: notes and their commitments, nullifiers, the spend proof, keys and addresses, and note encryption. It ends with what an observer can and cannot learn, the assumptions on which the construction rests. How private blocks are produced is the subject of the next section; how value enters and leaves the private chain is treated in Part IV.

4.1 Notes and commitments

A "note" is a shielded coin. It holds a value, the recipient's "address key" ak, and two secret numbers chosen by the sender, the note's "randomness": drawn at random for a transfer, and derived from the wallet's seed for a deposit. A note Alice receives might hold 10 XSS, her address key and two random numbers that no one else knows.

Notes never appear on the ledger. In their place the ledger records each note's "commitment": a hash of the note's value, its address key and both of its random numbers. The hash function is Rescue-Prime, which is designed to be cheap to evaluate inside a STARK proof. A hash acts as a fingerprint: it is easy to compute from its input, and infeasible to invert or to match with a second input. The randomness is what makes the commitment private. Without it, an observer who suspected that Alice had received 10 XSS could compute the commitment of that note and compare. With two unknown random numbers in the input, there is nothing to guess against, and the commitment reveals nothing about the value or the recipient. Because no second input produces the same hash, a commitment also fixes its note: once published, it can be opened in only one way.

Commitments are collected in the "commitment tree", an append-only Merkle tree of fixed depth. A Merkle tree hashes its leaves in pairs, then hashes those results in pairs, and so on up to a single "root". Whoever knows a leaf and the short path of sibling hashes from that leaf to the root can prove that the leaf is in the tree, and anyone who holds the root can check that proof. In the commitment tree every inner node is a Rescue-Prime hash of its two children, and every empty position holds the hash of a fixed empty leaf, so the root is defined however full the tree is. Each accepted transfer appends its two new commitments as leaves.

All of this arithmetic takes place in a prime field: numbers are integers modulo a fixed prime, and a sum that passes the prime wraps around to a small number, as $7 + 8$ reads 3 on a twelve-hour clock. The shielded pool works over a single field of this kind, and Section 4.3 shows why the wrap-around matters for value.

4.2 Spending without pointing: nullifiers

To stop a note from being spent twice, the chain has to know when a note has been spent. The obvious way, naming the spent note's commitment, would link the payment that created the note to the payment that spends it, and those links are exactly what privacy must hide.

Instead, spending a note reveals its "nullifier". The nullifier is a hash of the owner's secret "nullifier key" nk, the first of the note's two random numbers and the note's position in the commitment tree. It has three properties that together solve the problem. The same note always yields the same nullifier, so a second spend is recognisable. Only the holder of nk can compute it. And nothing in it points to the commitment it cancels, so an observer learns that some note was spent, but not which.

Every node keeps the complete set of nullifiers that have been spent, and a transfer that reveals a nullifier already in that set is rejected. This works only if each note has ex-

actly one nullifier; otherwise the owner could spend the same note under several. The spend proof closes that gap: it ties the position used in the nullifier to the bits of the Merkle path that proves the note is in the tree, so the position cannot be varied; it binds the randomness used in the nullifier to the randomness in the note's own leaf; and it recomputes the address key from nk, binding nk to the nullifier and the address key to the spent leaf. Only the key that the note's address commits to can therefore produce the note's nullifier, and it can produce only one.

4.3 The spend proof

A shielded transfer is carried in an "action". An action consists of a public statement, one proof, an anti-spam proof of work described in Section 4.7, and, optionally, the new notes encrypted for their recipients. It carries no signature and no fee. An action spends two notes and creates two. A payer who holds only one note cannot pay until it holds two: the wallet refuses and asks first to consolidate or to shield more. There is no dummy note.

The proof is a STARK, a zero-knowledge proof: it convinces anyone who checks it that a statement is true while revealing nothing about the secrets that make it true. For an action, the proof shows that the prover knows two notes and their keys such that:

1. both notes' commitments lie in the commitment tree under the same public root, the action's "anchor";
2. the published nullifiers are the nullifiers of those two notes, computed with the key that each note's address commits to;
3. each published output commitment is a correctly formed commitment to its output note;
4. the two input values sum to the two output values;
5. every value is below a fixed bound.

Value conservation is item 4: if the inputs hold v_1 and v_2 and the outputs hold v_3 and v_4, the proof enforces

$$v_1 + v_2 = v_3 + v_4,$$

with each input value read from its tree leaf and each output value from its commitment. There is no fee term, so inputs equal outputs exactly. Item 5 is what makes item 4 mean what it says. In a prime field, an equation can balance by wrapping around: an output of a huge value can make the sum wrap back to the input total, creating money from nothing. The spend proof's rules are polynomial equations over the prime field of $p = 2^{64} - 2^{32} + 1$, and the proof breaks every value into its bits and requires it to lie below 2^{62}, so that a sum of two values stays below 2^{63}, far below the prime, and no sum can reach the prime and wrap. No separate range proof is needed.

An example shows what this achieves. Alice holds two notes worth 6 and 4 XSS and pays Bob 7. Her action spends both notes and creates a note of 7 XSS for Bob's address key and a note of 3 XSS for her own, the change. The chain sees the anchor, two nullifiers, two new commitments, the network identifier and the expiry, the encrypted notes, the proof of work and the proof. The proof establishes that $6 + 4 = 7 + 3$, and none of these numbers appears anywhere.

A proof only shows that the notes are in the tree under the anchor it names, so which roots may serve as anchors matters. At each private-block boundary, the tree root at that moment is recorded as a valid anchor; roots from the middle of a block never become anchors, so every node agrees on which anchors exist. A spend may anchor to any root in a window of the last 1,280 recorded roots, which spans at least the last 1,280 private blocks. The window lets a wallet build its proof against a root it already knows while the tree keeps growing, and the proof remains valid while the action travels to the producer. A note newly minted into the pool becomes spendable once the next block boundary records a root that contains it.

Two further values are bound into the proof: the network identifier and an expiry. The proof's transcript absorbs both, so neither can be altered without invalidating the proof. A node rejects an action made for another network, and one whose expiry lies below the main-chain height to which its private block is bound.

A node accepts an action only if all of the following hold:

1. its anchor is a recorded block-boundary root that the node still retains;

2. its two nullifiers differ from each other;
3. neither nullifier is already in the spent set, nor revealed earlier in the same private block;
4. its proof verifies against its statement;
5. its network identifier is the node's own, and its expiry has not passed;
6. its proof of work is valid and recent enough (Section 4.7).

The transfers of a private block are applied all or nothing: every proof is verified and every statement checked against the state after the block's deposit mints and burns, and only then does any transfer change the state.

A STARK is zero knowledge in the following sense. The verifier checks the proof by opening a limited number of positions in the columns of values that the prover computed. Each auxiliary column ends in a tail of fresh random rows, longer than the number of openings a verifier can make, so the openings do not determine the prover's secrets. A STARK also uses no elliptic curve and no other structure that a quantum computer breaks: it is built from hash functions, and the STARK circuits of the shielded pool are post-quantum secure.

4.4 Keys and addresses

A wallet's keys all descend from one secret, the "spending seed", and each is derived under its own domain tag, a fixed label hashed in with the input so that keys derived for different purposes can never coincide.

Nullifier key `nk` Derived from the spending seed. It carries spend authority: whoever knows it can compute nullifiers and prove spends.

Address key `ak` The hash of `nk`. Because the hash is one-way, a published address key commits to the nullifier key without revealing it.

Note-decryption secret Derived from the spending seed under its own domain, independently of `nk`, so that holding it gives no spend authority.

Incoming viewing key The address key together with the note-decryption secret. It can detect and decrypt received notes, and it contains no nullifier key.

Outgoing viewing key Derived from the spending seed under its own domain. It lets a sender recover, and disclose selectively, the payments they have made, without gaining spend authority.

A shielded receiving address is a pair: the address key, which is bound into the commitments of notes paid to the address, and an encryption public key, which is used only to encrypt notes for delivery. Written out, an address is a checksummed bech32m string that begins with `xss`.

Spend authority is therefore knowledge of `nk`, and nothing else. When Alice spends, she signs nothing; her proof shows that she knows a key whose hash is the address key inside the notes she spends. An attacker who wants her notes must find a preimage of that hash, which is infeasible for a classical and a quantum adversary alike.

Commitments, nullifiers and address keys are hashed under distinct domain tags, so their inputs come from disjoint domains by construction. A value computed as one of them can never be presented as another.

4.5 Delivering notes

Bob cannot spend a note he does not know. To spend his 7 XSS, he needs the value and the randomness, since his own address key is the third ingredient of the commitment. The chain itself is the delivery channel.

Shielded note encryption uses ML-KEM-768, a key-encapsulation mechanism (KEM). A KEM lets a sender derive a fresh secret key together with a short ciphertext that only the holder of the matching private key can turn back into the same secret. Alice encapsulates a fresh key to Bob's encryption public key and encrypts under it the note's value, its randomness and a free-form memo. ML-KEM is standardised for post-quantum use, so a ciphertext harvested from the ledger today remains unreadable once a quantum computer exists.

The ledger carries no tag that says whom a ciphertext is for. Bob finds his notes by trying to decrypt every ciphertext with his note-decryption secret; the ones that decrypt are his. Each note is encrypted with an authenticated cipher, so a ciphertext decrypts only when its authentication tag verifies under the key Bob's secret recovers, and every other ci-

phertext fails that check. An observer sees ciphertexts and nothing that points to their recipients. Bob's wallet does not need the private chain for this: it obtains the ciphertexts, together with the anchors and Merkle paths it needs to spend, from its gateway (Section 4.9) by RPC (remote procedure call) over Tor. The encrypted notes are optional in an action, but the wallet always attaches both, so recipients find their notes by scanning. Without them, a recipient learns of a payment only outside the chain, from the payment disclosure (Section 4.6). Scanning is the recipient's side of carrying no tag: because nothing marks whom a ciphertext is for, a wallet tries every one, and its work grows with the activity of the whole pool.

A relay that passes an action along might try to damage delivery by altering a ciphertext, leaving the payment valid but unreadable. It cannot. A STARK is made non-interactive by the Fiat–Shamir method, in which the verifier's random challenges are derived by hashing a transcript of everything the proof commits to. The two output ciphertexts are absorbed into that transcript, so any alteration changes the challenges and invalidates the proof.

4.6 Disclosing one payment

Privacy by default raises a practical question: how does Alice show that she paid Bob, to Bob or to an auditor, without opening her whole wallet? For each output, a sender can seal the note's opening, meaning its value, address key and randomness, under a key derived from the outgoing viewing key. Alice can later disclose the opening of one payment. The verifier recomputes the commitment from the opening and checks that it is in the commitment tree. Because a commitment has only one opening, a disclosure proves that one payment, of that value to that address key, and nothing about any other note.

4.7 Feeless transfers and fework

Shielded transfers are feeless. A fee is also what makes spam expensive, so a feeless system needs another cost. Instead of a fee, every action carries a "fework": a proof of work computed with Argon2d over the action's identity and the hash of a recent main-chain block. Each Argon2d evaluation here uses a small, fixed amount of memory, and the cost of the fework comes mainly from the many evaluations that its difficulty requires, not from a large memory footprint.

The fework is bound to its action and to its time. It is computed over the action's identity, so work done for one action cannot be moved to another. And the main-chain block it references may lie no more than a small, fixed number of blocks behind the height to which the including private block is bound. Stale work must be recomputed, so work cannot be stockpiled in advance. The referenced block is public, so it also places an action's creation within that short window; this is the price of work that cannot be stockpiled.

Within a private block, actions are ordered by their identity hash, not by the order in which they arrived, so the order does not depend on when each node happened to receive them.

4.8 Entering and leaving the pool

Value enters the private chain by minting and leaves it by burning; Part IV explains how the main chain locks and releases the matching XST. What matters here is what each step shows. When a note is minted, its amount and its new commitment are public, while the recipient's address key and the note's randomness stay private. A minted note consumes no nullifier. When a note is burned, its amount, its nullifier, the anchor of its proof and the script that receives the payout are public, while which note was burned stays hidden. The amounts at the boundary are public because the main chain locks and releases exactly those amounts. A distinctive amount that enters and the same amount leaving shortly afterwards can therefore be matched by an observer, and the protection against this lies in the user's choice of amounts and timing.

Shielded transfers and burns record their nullifiers in the same set. A note is therefore spent exactly once, either in a transfer or as an exit, never both.

4.9 What an observer learns

Table 1 collects what the ledger shows and what it hides.

The anonymity of a spend follows from the proof. A nullifier points to no commitment, and the proof shows only that the spent notes lie somewhere in the tree under the anchor. A spent note could therefore be any note in the tree at that anchor.

The holder of an incoming viewing key sees more than an observer, but less than the owner. Computing a note's nullifier requires nk, which an incoming viewing key does not contain, so its holder sees notes arrive but cannot tell whether they have been spent. It therefore cannot show a balance on its own, since telling spent notes apart would take the nullifier key, which is spend authority.

The ledger is not the only place where a payment can be observed; the network that carries it can reveal where it came from. By default a wallet reaches the gateway run by the Stealth team over Tor, at the gateway's onion address, an address that exists only inside the Tor network, and the gateway relays payments into the network over Tor. Neither the gateway nor any node therefore learns the IP (Internet Protocol) address of a payment's sender. A user who runs a staker node can point the wallet at that node instead.

The gateway itself sees more than the network does. A wallet asks its gateway for the position and the Merkle path of each of its notes, one at a time, so the gateway learns which notes the wallet holds; Tor hides only the wallet's IP address. A user who runs their own staker node and points the wallet at it avoids this, because the wallet then asks only that node.

4.10 Security and assumptions

The construction rests on four assumptions.

A1. Hash functions. The domain-tagged Rescue-Prime hash is collision resistant and one-way, and its output reveals nothing about an input that contains fresh randomness; the BLAKE3 hash of the STARK transcripts is collision resistant.

A2. Proof system. The STARK is sound, so a proof of a false statement is accepted only with negligible probability, and zero knowledge, so a proof reveals nothing about the prover's secret inputs beyond the truth of its statement.

A3. Encryption. ML-KEM-768 and the note encryption built on it keep plaintexts confidential, also against a quantum adversary.

A4. Feework. No one can produce valid feework for an action without doing the work.

Under these assumptions the shielded pool has the following properties.

No double spend. Each note has exactly one nullifier, because the proof fixes the position, the randomness and the key that go into it. Every node keeps every spent nullifier and rejects a nullifier already spent, a nullifier revealed twice in one block, and an action whose two nullifiers are equal. Transfers and exits share the one set. The set and the commitment tree only grow: every transfer adds two nullifiers and two leaves for good, and no retention policy can prune them, because any unspent note may still be spent and any spent note must stay spent.

No inflation. Every action conserves value exactly. The spend proof of Section 4.3, a STARK, checks this with polynomial equations over the prime field of $p = 2^{64} - 2^{32} + 1$, and it holds every value below 2^{62}, so no sum can balance by wrapping around. Within the pool, value can move but never grow.

No theft. Spending a note requires the nullifier key to which its address key commits. The address key is public and the nullifier key is its hash preimage, so knowing an address gives no power to spend from it. The incoming and outgoing viewing keys contain no nullifier key and give none. The note-decryption secret is derived independently of nk, so decrypting a note gives no spend authority either.

No tampering. The output ciphertexts, the network identifier and the expiry are all bound into the proof. An action cannot be moved to another network, kept past its expiry, or altered in transit so that its recipient cannot read it.

	Public	Hidden
Transfer	that an action took place; its anchor, two nullifiers and two new commitments; its network identifier and expiry height; the encrypted notes; the proof; the feework and the main-chain height it references	sender, recipient and every value; which notes were spent; any link between the transfer and the transfers that created its inputs
Entering	the amount; the new commitment	the recipient's address key; the note's randomness
Leaving	the amount; the nullifier; the anchor; the payout script	which note was burned

Table 1: *What the ledger shows about shielded activity, and what it hides.*

Privacy. Commitments hide their notes (A1), nullifiers do not point to commitments, the proof reveals nothing beyond its statement (A2), and the encrypted notes carry no recipient tag and stay unreadable to all but their recipients (A3). What remains public is listed in Table 1.

Quantum security. None of these guarantees depends on elliptic curves. Spend authority and soundness rest on hash functions, against which a quantum computer gains only Grover's square-root speed-up, and confidentiality rests on a post-quantum KEM.

5 The sidechain architecture

Section 4 described the shielded actions that private blocks carry. A chain of private blocks needs three things: a producer for each block, agreement among all nodes on which block that is, and a way for every node to obtain it. Four facts make this harder than it looks. If a node that is not a block's producer builds its own version of the block from the actions it happens to hold, the result can differ from the producer's, and the node has diverged. If the producer's block for a slot never reaches the network and every node waits for it, every node stops at that height, and the minting of bridged value stops with it. If a staker replaces its signing key, the staker registry keeps only the new key, so the registry alone cannot check blocks signed before the change. And if anyone can download the private chain, anyone can keep a full copy of it. This section describes how the private chain is produced, ordered, checked, distributed and reorganised in answer to these four facts.

5.1 Private blocks

Private blocks form their own hash chain: each names its predecessor. A private block commits to the anchor, nullifiers and output commitments of each of its ordered shielded actions and to the network identifier, amount, anchor, nullifier and payout script of each bridge burn it contains, and its header commits to the roots of the note tree and of the burn tree as they stand after the block. The block's hash is a domain-separated Rescue-Prime hash, the hash of Section 4, over all header fields, and those fields include the hash of the main-chain block to which the private block is bound. The private block at each height therefore fixes, in one value, the action and burn fields it commits to, its resulting state, its place in the private chain and the main-chain block it belongs to.

In turn, the main chain commits to every private block: a later main-chain block carries the private block's hash, and every node checks it. Each chain thus commits to the other, so once that later main-chain block is on the chain, neither a main-chain block nor a private block can be paired with a different partner, even when the two travel separately.

5.2 Who produces a private block

The producer of a private block is the qPoS staker that produced the main-chain block to which it is bound. That producer builds the private block from the actions in its own pool and signs it; no other node builds a private

block from its own pool. For each private height, the finality committee then decides either the producer's block or the fallback block, which every node builds identically from the main chain alone. Every node applies only the decided block. If the producer's block never reaches the committee, the committee decides the fallback block, and neither the private chain nor the minting of bridged value stops.

Suppose Carol, a staker, produces main-chain block 500, to which private block 120 is bound. Carol builds private block 120 from her pool, signs it and sends it out. Dave, another staker, holds a slightly different set of actions in his pool, but he does not build a block from them; had he done so, his private block 120 would differ from Carol's. The committee decides Carol's block, and Dave's node applies it. Had Carol's block never reached the committee, the committee would have decided the fallback block for height 120, and every node, Carol's and Dave's included, would have built the same one from the main chain.

Because the main-chain block names the staker that signed it, only that staker can sign the bound private block. For the same reason, a private block bound to a main-chain block that is not a qPoS block names no producer, and it is refused. Every staking node runs the private chain, because each staker is the only producer of the private block of every main-chain block it makes.

5.3 One thread, one order

The node has one clock for consensus: a single thread orders all consensus events. That thread processes every consensus command one at a time, in the order in which the commands arrive. After a main-chain block connects, the same thread advances the private chain bound to it. Transaction admission runs on the same thread too, so no second writer can change the consensus state while the thread is working.

With one writer, the node never has to reconcile two views of its own state. The main chain and the private chain bound to it advance in one sequence, and each step sees the result of the step before it. A failure in the private chain is contained, and the main chain keeps running.

5.4 Checking a producer's key

To check a private block's signature, a node needs the key that its producer was entitled to sign with at that height. Stakers sign blocks with their "delegate key", the signing authority that the qPoS paper introduces; a private-block producer signs with its ML-DSA-65 delegate key. A staker's delegate key can be replaced. Replacing it overwrites the key in the staker registry, so a node that checks an old block against the registry would use the wrong key.

Each node therefore keeps a full-history index of delegate keys, the Delegate-Key History Index (DKHI). It records each change of a staker's ML-DSA-65 delegate key, one entry when the key is first set and one each time it is replaced, so it grows with key changes, not with blocks. A lookup takes the staker's latest entry before the bound main-chain block, which is the key in force as that block is entered and the key with which that block's own signature was checked. In the example, if Carol replaces her delegate key at main-chain block 520, a node that syncs private block 120 later still checks Carol's signature against the key she held entering main-chain block 500.

The DKHI never guesses. It advances block by block, or over a whole replayed range written in one step, and refuses any gap, so it never claims to cover a height it did not record, and a lookup fails closed unless the index covers the queried height: a block at a height the index does not cover is refused, never checked with a stale key. In a main-chain reorganisation, rows are removed only above the fork, and the new branch's rows are written in the same step, so the history below the fork is kept.

5.5 Adopting a received block

Blocks travel between nodes, but transport never decides authority: every received block passes through the same adoption checks, whatever peer it came from. A node adopts a received private block only if the finality committee has decided it for its height and all of the following hold:

1. the block is in its canonical encoding; the producer's signature covers its header

and every field of its actions and burns except their STARK proofs;

2. it is bound to a qPoS main-chain block, and that main-chain block is canonical at its own height;
3. the producer's signature verifies against the staker's delegate key as of the bound main-chain height;
4. its height, its predecessor and its bound main-chain hash are the ones the node expects for its next height;
5. every action carries valid feework, judged against the block's bound main-chain height, so that every node reaches the same verdict;
6. every action proof verifies before any of the block's actions changes the state;
7. the note-tree and burn-tree roots that the node derives itself after applying the block equal the roots the block commits to.

Rule 7 means that no node takes a producer's word for the resulting state: each receiver re-derives it.

Two further rules keep a node's store unambiguous. A node never stores or serves two different bodies at one private height. And a node remembers a refused block by its exact bytes, so that refusing one block never blocks a different, valid block for the same height.

5.6 Synchronisation and recovery

Only verbatim producer-signed bytes move between nodes. A node announces its private tip and the lowest height it offers, and it never announces a block it cannot serve. When a peer announces its tip, a node requests blocks from its own next height, but only if the peer's advertised range covers that height. The private chain is one-to-one with main-chain heights, so sync is driven by height rather than by a search for a common ancestor. A serving node sends the signed bytes it stored, unchanged, and when it lacks a height it says so explicitly, naming the height it lacks and the range it can serve.

Suppose Erin's node holds the private chain up to height 450, and a peer announces a tip of 620 and a lowest height of 300. Erin's node asks for 451 onwards and checks each block as it arrives. Had the peer advertised a lowest height of 460, its announcement would trigger no request; but Erin's node also asks each of its peers for its next height at intervals, so it would still ask this peer for 451 and receive an explicit not-found, and any connected peer whose range covers 451 would serve it.

The same mechanism recovers a decided block that a node has missed. If the decided block is a producer's block, the node fetches it from a live peer that holds it. The fetched block is the producer's own signed bytes, so recovery involves no new derivation and no new authority, and the block passes the same adoption checks as any other. If the decided block is the fallback block, the node builds it itself.

Serving has limits that protect the server. Each peer is limited in how many private blocks it can make a node verify. And private-chain sync is open only to stakers: peers that are not stakers are refused, and the staker-only network requires mutual authentication between peers. The reason is the fourth fact above: on an open network, receiving the chain is the same as archiving it, and anyone who could sync the private chain could keep it. A user who wants a private-chain endpoint of their own therefore runs a staker node; a wallet needs none, since it obtains ciphertexts, anchors and Merkle paths from its gateway (Section 4.9).

5.7 Reorganisations

The private chain has no fork choice of its own; it follows the main chain. A node detects when a main-chain block to which its private chain was bound is no longer canonical, and the fork point for the private chain is then the newest private block whose bound main-chain hash is still canonical.

From the finality fork on, the private chain binds only final main-chain heights, so a bound private block cannot be reorganised away. Where a rewind is needed, it is automatic only when the orphaned private blocks carried no actions and no burns; anything else halts for a resynchronisation. And if a rewind is not provably safe, the private chain and custody stop, rather than bind main-chain blocks that may have been orphaned. Resynchronising a node that has halted this way is its operator's task.

5.8 Security and assumptions

The private chain's integrity follows from four facts established above. Only the staker named by the bound main-chain block can sign a private block, and its signature is checked against the key it held at that height. The signature covers the block's header and every field of its actions and burns except their STARK proofs. Every node re-verifies every proof and re-derives every root before it adopts a block. And every node orders these steps on one thread, after the main-chain block they depend on.

These rest on the following assumptions: that the producer's signature cannot be forged, so that no one else can sign in the producer's name; and that the node's main chain is canonical, which finality makes permanent from the finality fork on.

Part IV
Security

6 Post-quantum security

Section 1 described what a quantum computer breaks. The threat is not the same for every part of the protocol, and neither is the answer. Encryption protects confidentiality, and its quantum threat is harvest now, decrypt later: the exposure exists from the moment a ciphertext is transmitted. Signatures protect against forgery and theft; that threat exists only once a quantum computer exists, and it cannot be harvested after the fact. Block signatures are a case of their own. Neither the shielded chain nor commit-reveal routes around them, and a forged block signature attacks consensus itself, through a rewritten history or through conflicting blocks. This section gives each part of the protocol its route and explains why each route holds.

6.1 One route for each part

Post-quantum security in Stealth has three parts: ML-DSA-65 signatures on blocks and on finality votes, commit-reveal for transparent spends, and post-quantum note encryption with a KEM. Table 2 sets out which part of the protocol each one protects. No user-level post-quantum signature exists anywhere in the protocol.

A holder therefore chooses the protected route for their coins: the shielded chain, for privacy together with post-quantum safety, or commit-reveal, for a transparent spend that is safe against a quantum adversary.

6.2 The shielded chain

The shielded chain's spend authorization and its confidentiality are both post-quantum, and neither is a signature. A shielded address's address key is the Rescue-Prime hash of the nullifier key (Section 4), so shielded spend authority rests on a hash preimage. The STARK that proves a spend is hash-based and uses no signature that could be forged, and the STARK circuits are post-quantum secure as built. Shielded spend authority also does not depend on the scheme with which producers sign blocks; it rests on the nullifier key alone.

Confidentiality rests on ML-KEM-768. Note encryption uses this post-quantum KEM, and the wallet always attaches both encrypted notes, so harvesting shielded payments and messages gains an adversary nothing: a ciphertext recorded today stays unreadable once a quantum computer exists. Only the KEM keeps a hoarded stream of note ciphertexts unreadable. A post-quantum signature would not, because signatures address forgery and encryption addresses confidentiality.

The two layers are kept apart. The note-encryption secret is derived from the spending seed under its own domain tag, separately from the nullifier key that carries spend authority. The KEM ciphertext is not absorbed into the Rescue key-derivation function, and the address envelope that carries the KEM key is extensible.

6.3 Block signatures

qPoS block signatures are hybrid: consensus enforces both an ECDSA signature and an ML-DSA-65 signature, and every staker's registry entry holds an ML-DSA-65 delegate key. A qPoS block is valid only if its block signature verifies against the producing staker's registered delegate key. Private-chain producers sign with ML-DSA-65 in the same way, and a producer's private block can be decided and applied only with a producer signature that verifies against the producer's public key (Section 5); the fallback block carries no signature, because every node builds it itself.

Because consensus requires both signatures on a main-chain block, forging one requires breaking both schemes. A classical adversary faces ECDSA and ML-DSA-65; a quantum adversary who breaks ECDSA still faces ML-DSA-65. The ML-DSA-65 signature is made once per block, off the transaction path, so it adds no cost to any transaction. Its price is a second, larger signature in every main-chain block, paid once per block rather than once per payment.

6.4 Transparent spends

Transparent spends are authorized by secp256k1 ECDSA signatures, and they stay that way. ECDSA verification is permanent, all existing coins are grandfathered, and there

Part	Quantum threat	Route
Shielded chain	forged spends; harvested note ciphertexts	spend authority from a hash preimage, proved by a hash-based STARK; notes encrypted with ML-KEM-768
Blocks of both chains	a forged producer signature, which attacks consensus	every producer signs with ML-DSA-65; on the main chain together with ECDSA
Transparent coins	a private key derived from a revealed public key	commit-reveal (enclave) spends, anchored in finality

Table 2: *The quantum threat to each part of the protocol, and the route that answers it.*

is no forced migration and no freeze. Transparent spends carry no post-quantum signature: block space cannot absorb the size of one on every transaction, and the one compact alternative, Falcon, gives up side-channel safety.

Transparent coins are protected instead by a two-phase spend, commit-reveal. The Pq-Commit upgrade enables a protected script type, the enclave. A spend of an enclave coin is valid only if a commitment matching its public key, its "outpoint" (the reference that names the coin by the transaction that created it and its position there) and its destinations was included in a BFT-finalized block.

Suppose Alice holds 40 XST in an enclave address and wants to pay Bob 30. She first publishes a commitment to her public key, to that coin and to the destinations, 30 XST to Bob and 10 XST back to her as change, while her public key is still unknown to anyone. Once the commitment is in a final block, she spends the coin with an ordinary ECDSA signature, which reveals the public key. An attacker who derives her private key from it can sign a spend that sends the 40 XST to himself, but that spend matches no finalized commitment, and the chain rejects it. The attacker cannot front-run the pre-committed destination. The protected spend still carries only its ECDSA signature, and none of the size of a post-quantum one. An enclave address can be funded many times, but its first spend reveals its public key, so from then on every coin still at that address, and every coin sent to it later, is exposed; the spend's own change always goes to a fresh enclave address. Section 10 describes enclave addresses in full.

6.5 Security and assumptions

The routes rest on four assumptions.

- **Q1. Hash functions.** The hash functions of the protocol are one-way and collision resistant against a quantum adversary, who gains only Grover's square-root speed-up against them.
- **Q2. Encryption.** ML-KEM-768 keeps note plaintexts confidential against a quantum adversary.
- **Q3. Signatures.** ML-DSA-65 signatures cannot be forged by a quantum adversary.
- **Q4. Finality.** Less than one third of the finality committee's weight is Byzantine (Section 7).

Under Q1 and Q2 the shielded chain is unforgeable and confidential against a quantum adversary. Under Q3 no block of either chain can be forged. Under Q1 and Q4 an attacker who derives the key cannot redirect a coin in an enclave address whose key is unexposed: until the reveal only a hash of its public key is known, and every valid spend must match a commitment in a finalized block. What remains is the finality lag between the reveal's confirmation and its finalization (Section 10). Protection of a transparent coin is its holder's choice, since no coin is forced to move: a coin never moved into an enclave address or into the shielded chain stays protected by ECDSA alone.

7 BFT finality

Without finality, qPoS settlement is probabilistic. A block becomes harder to reverse as blocks accumulate on top of it, but a rule that treats a block as settled after some depth is a heuristic, not a guarantee. BFT finality turns probabilistic settlement into deterministic finality. A committee of stakers votes on each block, weighted as in qPoS, and a block that members holding more than two thirds of the committee's weight have ratified is final and is never reverted. Finality is part of consensus: it takes part in the fork-choice rule, and it is not a checkpoint service laid over the chain. Its security argument rests on mechanism alone: the threshold, the weighting, partition safety and slashing.

7.1 The committee

The finality committee is the set of enabled qPoS stakers, each with its qPoS weight, and the threshold is a share of that weight, never a head count. A staker's weight grows with the blocks it has produced and with its seniority, and every staker is bought at a price, set for each block, that rises with each staker registered, so three stakers bought in successive blocks cost more than three times the first.

The committee changes as stakers are enabled, disabled and produce blocks, so it is frozen once per "epoch", a fixed 512 blocks. At each epoch boundary block, the committee (its enabled members, their weights and their keys) is frozen from the state before that block. The boundary block carries exactly one commitment to the "committee root", a Merkle root over one leaf per member that holds the member's identity, its ML-DSA-65 delegate key and its weight; blocks off the boundary carry none. Every node recomputes the committee root from its own state before the block and rejects a boundary block whose committed root differs.

Every height above the finalized frontier is tallied against the committee of the epoch that contains the current finalized frontier. That frontier lies in the prefix of the chain that is never reverted and that all honest nodes share, so all honest nodes tally against the same committee.

7.2 Votes

When a node connects a block, it casts a "ratifying vote" for it if the node is a member of the committee derived from the state before the block. The vote is an ML-DSA-65 signature over a domain-separated digest of the block's hash, so a producer's block signature can never count as a ratifying vote. The qPoS block hash commits to the block's height and to its producer, so a vote over a block hash also commits to that height.

A vote counts only if its signer is a distinct committee member and the signer's ML-DSA-65 delegate key verifies it. A height's signer set consists of its own direct ratifiers: a vote for a later block does not count for an earlier one, and the producer counts only through its own ratifying vote.

An honest member never votes twice at one height. Each validator persists a "sign-once record" before it votes. The record refuses a vote at a height below the highest the validator has already ratified, and it refuses a second, different vote at a height already ratified.

7.3 When a block is final

The finality rule is a single inequality. Let W be the committee's total weight.

1. A block is final when the weight of the committee members who ratified it strictly exceeds $\lfloor 2W/3 \rfloor$.
2. A block is super-final when members holding the full committee weight have ratified it.
3. The finalized frontier advances one height at a time: a height becomes final only when it is the next height after the frontier and has gathered its own quorum.

Suppose the committee has four members, Carol with weight 5, Dave with 3, Erin with 2 and Frank with 2, so that $W = 12$ and $\lfloor 2W/3 \rfloor = 8$. If Carol and Dave ratify a block, their weight is 8, which does not exceed 8, and the block is not final. If Erin ratifies it as well, the weight is 10 and the block is final.

A block's finality certificate is the list of its members' ML-DSA-65 ratifying signatures. The signatures travel with the block but outside its header preimage, so attaching them or pruning them never changes the block's

identity. A node that receives a copy of a block with more ratifying signatures adopts it and passes it on; a copy with fewer is not a downgrade. A committee larger than the cap on ratifying signatures is not frozen at its epoch boundary; finality halts instead, and fails closed.

7.4 Fork choice

A node never reorganises past a final block. A branch that would revert a final block is rejected permanently, not merely disfavoured: a reorganisation whose fork point lies below the finalized height is rejected whatever its weight or depth. The standard qPoS fork choice applies only within the suffix above the highest final block; within that suffix it is lock-aware, a reorganisation is taken to a heavier chain, and between equally heavy chains a tie-break sourced from finality decides.

7.5 Why a final block stays final

A final block is never reorganised away, under the assumption that less than one third of the committee's weight is Byzantine. The reason is that two quorums always overlap. Let A and B be the sets of members that ratified two conflicting blocks at the same height, each with weight above $\lfloor 2W/3 \rfloor$ and therefore above $2W/3$. Then

$$\begin{aligned} w(A \cap B) &\geq w(A) + w(B) - W \\ &> \tfrac{2W}{3} + \tfrac{2W}{3} - W = \tfrac{W}{3}. \end{aligned}$$

Both blocks could become final only if members holding more than a third of the weight signed both, which is slashable equivocation and is excluded by the premise. In the example, any two sets of weight above 8 overlap in weight above 4: the sets {Carol, Dave, Erin} and {Carol, Erin, Frank} share Carol and Erin, with weight 7.

The same inequality gives partition safety, which is the primary correctness property. If the network splits, at most one side can finalize, because two disjoint sets that each hold more than two thirds of the weight cannot exist. Under a split, finality stalls; it never finalizes two conflicting blocks.

7.6 Slashing

A member that signs two conflicting blocks at one height commits a slashable offence: nodes collect the two signatures as evidence, block producers put that evidence on the chain, and the offender is penalised as described below. There is no public path for submitting evidence. Only same-height double ratification is slashable. Conflicts across heights are removed by prevention instead, because a height becomes final only from its own direct votes and only after the height before it.

Evidence carries the header preimage of each of the two blocks, and the verifier recomputes both block hashes from them, so the evidence needs no lookup in the chain. Every validating node re-verifies on-chain evidence when it connects the block that carries it, and a block with invalid evidence is rejected. Evidence is valid only if all of the following hold:

1. both sides are qPoS headers, whose hashes commit to the height, so the offence height is authenticated;
2. both sides name the same height;
3. the two sides recompute to different blocks; two sides that recompute to the same block are a duplicate, not a conflict;
4. the declared committee epoch does not exceed the epoch of the offence height;
5. the accused is a member of the committee of the declared epoch;
6. both signatures verify under the accused's key, resolved as of the offence height, against the ratify digest.

Rule 6 has two consequences. A later key change cannot alter the verdict. And because a block signature never satisfies the ratify digest, an honest producer cannot be framed with its own block signature.

An offence is identified by the pair of signer and height, so one equivocation is penalised at most once. The penalty takes nothing from the staker. A staker has no bond: the price paid for it is burned at purchase. The penalty acts on the staker's rewards and, for a repeated offence, on its standing:

1. the first proven "double-sign", a signature on two conflicting blocks at one height, stops the staker's rewards for a

fixed week, and after that until its owner confirms with an operation signed by the owner's key; meanwhile the staker keeps producing blocks and voting;

2. further double-signs before the owner has confirmed count as the same incident, and each only restarts the week;
3. a double-sign after the owner has confirmed disqualifies the staker permanently.

The record of the offence stays with the staker. A first offence therefore costs rewards and requires the owner's acknowledgement; only a double-sign after that acknowledgement removes the staker.

Offences are found, not only reported. When a height finalizes, each node takes any member that ratified two or more different blocks at that height as an equivocation candidate.

7.7 Stalls and recovery

If more than a third of the weight is absent, the finalized frontier freezes, and no height finalizes below the strict bar, while blocks keep being produced. When the frontier stalls for longer than the node's finality window, at least 1,024 blocks, while the tip keeps rising, the node latches a halt: finality stops making progress, and blocks keep connecting. Shielded settlement is paced to the finalized frontier, so a stall also pauses shielded transfers, bridge-ins and bridge-outs, while transparent payments keep confirming; the private chain binds only final main-chain blocks so that nothing it applies can be reorganised away (Section 5).

When the quorum returns after such an outage, finality does not resume automatically. An explicit operator command re-arms it, after confirmation outside the protocol that the partition has healed, and the command acts only on a halted node. Resuming cannot weaken safety, because it only re-runs the unchanged strict tally above $\lfloor 2W/3 \rfloor$. An outage longer than a fixed recovery budget is not re-armed in place: the node stays halted until it is re-synchronised from an archive peer, and it then catches up from certificates as follows.

A node that re-synchronises advances its finalized frontier from the certificates that its peers have stored, each checked against the committee that finalized its height. It accepts a certificate only if it names the node's own canonical block at that height and its signatures clear the bar for that committee. It applies only the certificate for the height directly above its frontier; if that certificate is missing or fails, catch-up stalls and requests it, and it never skips ahead.

8 Consensus custody and the XST $\leftrightarrow$ XSS bridge

Value moves between the two chains through a bridge, and a bridge needs a custodian for the coins it holds. A committee that signs releases is the usual answer, and it has two failure modes: a release freezes when enough signers are lost, and it can be stolen by capturing the committee. The release of a shielded coin also has requirements of its own. The transparent chain must be sure that the shielded coin was burned, and it must pay the recipient named in the burn, without carrying the large burn proof and without any committee signature. And the shielded chain advances by decision: for each private height the finality committee decides the producer's block or the fallback block, and a node applies only the decided block, so a node that is behind lacks blocks that another node has already applied. If each node validated a release against its own shielded anchors, two honest nodes at the same main-chain height could reach different verdicts, and the chain would fork.

Stealth answers these with "consensus custody": custody of bridged XST is a rule of the chain. There is no committee, no bond and no signing; the chain verifies each burn. The custodian is the chain's honest majority, the same trust already placed in every block, and custody therefore requires BFT finality. The bridge has three boundary events: shield-in locks XST against a mint proof, transfers inside the shielded chain stay there, and unshield-out releases XST to the recipient bound in the burn.

8.1 Shield-in

On shield-in, XST is locked into a pool managed by consensus, against a mint proof that ties the amount to a note commitment. The main chain enforces the following:

1. a deposit below the minimum custody deposit is rejected;
2. every deposit names exactly one companion note commitment, the note to be minted, and a deposit without one is rejected;
3. at admission, consensus verifies a STARK proof that the companion commitment opens to the locked amount, running the same mint verifier as the shielded chain, so a deposit whose commitment could never mint is rejected before its funds move;
4. a note commitment binds to exactly one deposit, and a repeated commitment is rejected.

A deposit waits no fixed number of confirmations. Once the block that includes it is final, consensus locks it in place as a custody "Lock" output, with no further transaction, and with finality active its note is minted within one block, at most two. Lock outputs are exempt from script verification and can be spent only through the custody release rule; a transaction that spends one without being a valid release is rejected, and no Lock output can be spent in the block that creates it.

Locked deposits are minted in the shielded chain in a canonical order, so every node appends identical leaves and derives identical roots. From the finality fork on, private blocks bind only final main-chain heights, so a deposit mints only after its main-chain block is final. The shielded chain re-verifies, unconditionally, the proof that binds the note's value to the locked amount before it mints. A locked output can be minted only once, and a minted amount must lie inside the proof system's provable range, so that no value can wrap around the field. A locked deposit without a valid mint proof stalls the private block rather than being skipped, so that no locked deposit is ever passed over.

8.2 Burns

Unshielding begins in the shielded chain with a burn. A burn needs a STARK proof that a note worth exactly the burned amount exists under the anchor, is spent with the published nullifier and is bound to the recipient, so the destination cannot be redirected. The shielded chain accepts a burn only against a known block-boundary anchor, only if its nullifier is not already spent, and only if it names a standard recipient script, so that every accepted burn can actually be paid out.

Applying a burn builds a "receipt leaf" from the amount, a commitment to the recipient and the nullifier. The receipt is appended to a separate, append-only "burn tree", and the nullifier is recorded in the same atomic step. The burn tree uses its own empty-leaf domain, so a burn root can never equal a note-tree anchor.

8.3 Committing the shielded roots

The main chain has to know the shielded state in order to release against it, and it learns that state through finality. Each main-chain block may carry at most one "root commitment", a zero-value marker that records the block's own height and the roots of the note tree and the burn tree after the latest private block that the block's producer has applied. Block connection does not validate the committed roots; finality does. A validator ratifies a block only if it has reproduced both committed roots from its own shielded state. It abstains when the commitment names a different height. Abstaining casts no vote and does not use up the validator's one vote at that height, so a validator that was behind can still sign later. Leaving the roots to finality keeps a shielded stall from halting the main chain, which it would do if main-chain acceptance waited for shielded progress.

The legitimacy of a committed root therefore comes from the finality quorum, not from per-node re-derivation and not from trust. An honest node never derives a fabricated root, because it re-verifies every shielded proof and re-mints deposits from its own main chain. A stalled honest validator only abstains, which lowers the signing weight and can never push a fabricated root over the bar. And at most

one root is ever finalized per height, because two quorums always overlap (Section 7).

8.4 Release

A release pays out an unshield on the main chain. It carries a small "burn receipt": a Merkle branch that proves the burn is a leaf under a burn root that is committed on the chain and final. A release may cite any buried, final burn root; there is no maximum age, because the check reads only the main chain's hash index, which pruning never deletes. A release is valid only if all of the following hold:

1. it carries exactly one zero-value release marker, which holds the receipt, and no other custody marker;
2. every input spends a live custody Lock, and the value comes from the custody registry, not from the carried data;
3. it has exactly one recipient payout, and a second value-bearing output is rejected as a leak of value;
4. it re-locks change into at most one new Lock output, and that output holds more than zero; a Lock has no dust floor, so the least change is one atomic unit, 0.000001 XST;
5. the payout equals the burned amount in the receipt;
6. the verifier computes the receipt leaf itself from the amount, the recipient and the nullifier, taking the recipient from the payout output's own script; the receipt carries no leaf of its own;
7. a Rescue-Prime branch of fixed depth leads from that leaf to a burn root;
8. the release cites the block that committed that root, and that block's certificate is checked against the committee that certified it; a release whose committee the node does not know is refused;
9. the certified block hash, recomputed from the block's header preimage, matches the certificate, and the certificate shows finalizing weight;
10. the burn root is read from a private-chain block header that the private-block committee certified, and it equals the root the receipt branch reproduces;
11. the certified block is buried at or below the node's own finalized height, and its hash equals the node's own canonical block hash at that height;
12. the burn's nullifier has not been released before; the custody registry keeps the set of released nullifiers, and two releases of one nullifier in the same block are rejected.

A validating node that cannot run this verifier rejects the release. Lock outputs are exempt from script verification, so the release verifier is the sole authorization for spending them, and it has to be complete on its own. Rule 6 is what binds the payout to the burn: without it, a verifier would prove that a burn happened but not where the money goes. Rule 11 defeats certificates forged for blocks outside the chain, including forgeries made with the keys of a retired committee.

Applying a release marks each consumed deposit as released and records the nullifier, and the change becomes a new custody Lock that can itself be released later. Continuing the example of Section 1, Alice's 60 XST sit in two Locks. Bob's release spends one of them, pays 25 XST to the address committed in his burn and re-locks 5 XST as change. A release pays no fee: its inputs equal its outputs.

8.5 Conservation

The bridge keeps one invariant:

$$\text{minted} - \text{burned} = \text{locked},$$
$$\text{minted} \geq \text{burned}.$$

Shielded supply grows only through a mint backed by a locked deposit, at exactly one place, and a burn that would break the invariant is rejected before any state changes. On the main chain, a release's payout plus its change equals the locked value it consumes, and custody markers carry zero value, so a marker cannot shrink a transaction's outputs and create coins from nothing. In the example, 60 XSS were minted and 25 burned, and 35 XST remain locked.

8.6 Divergence

A private block's header commits to the roots of the note tree and the burn tree after the block. A receiving node compares the roots it derives itself with the roots the producer committed. On a mismatch it neither adopts the block nor rewinds itself: it stalls, and the finality committee's decision gives the verdict on which block the height receives. The node applies only the decided block. A block's roots become valid spend anchors only when the node adopts the block after matching them. A follower never trusts an unfinalized producer root over its own deterministic derivation.

The only definitive sign that a node has diverged is a decided block whose roots it cannot reproduce, and a Byzantine producer cannot forge that sign. A correctly synced node always reproduces the roots of a decided block, so it is never falsely marked as diverged. A node whose state disagrees with a decided block is held until it resynchronises, and shielded state that a node has applied is never rewound.

The main chain's own reorganisations are handled as well. A main-chain reorganisation restores the custody registry to its state before the reverted blocks, un-recording their deposits, locks and releases. A burn receipt already published in a block-boundary root is never silently undone, and a finalized root cannot be reorganised out from under an admitted release.

8.7 Security and assumptions

Custody safety rests on three things: the root commitment, the barrier that a validator reproduces a root before it ratifies it, and finality's ratification. Its assumption is finality's own: less than one third of the committee's weight is Byzantine. Forging a finalized fabricated root needs more than two thirds of the committee's weight to be Byzantine, which lies beyond that assumption. A root is certified rather than trusted because a fabricated note under a trusted root would pass the burn check and drain the whole locked pool; and a release does not re-run the burn's STARK proof, because that proof shows membership only under a supplied anchor, whose legitimacy comes from the finality quorum either way.

Each attack meets a check. A forged proof fails verification; a replayed nullifier is refused; a release that does not conserve value is refused; a stale or foreign anchor is refused; and a redirected recipient breaks the binding between the proof and the payout. Verification works over an integer prime field, so every node computes a bit-identical result. Releases wait for finality. If too little weight has caught up on the shielded chain to finalize a root, releases against it wait and the funds stay locked, never lost, while the main chain keeps advancing, so a correlated shielded stall of more than a third of the weight costs liveness, not safety. A producer that commits a wrong root cannot get it finalized, but while its block stays on the chain unratified, it can hold the finality frontier back.

Part V
Applications

9 Private messenger

A blockchain can carry messages as well as payments, and on most chains the two are priced alike. Where every transaction pays a fee, every message pays one: a conversation of a hundred short messages costs a hundred fees, however little each message says. A message written into a public ledger is also readable by everyone, and it is kept for as long as the ledger exists. Encrypting it hides the text but not the permanence. The ciphertext stays on the ledger, and if the encryption rests on elliptic curves, an adversary who records it today can read it once a large quantum computer exists.

A private conversation needs four things that a ledger does not give by default. It should pay no fee per message, so that a conversation is not priced by how many messages it holds. It should be readable by its participants and by no one else, now and after a quantum computer exists. It should not reveal who is talking to whom. And a message should be able to disappear: a ledger is built never to forget, while a conversation is meant to be read and then let go. Two further facts constrain any design that keeps messages on a network of nodes. On an open network, receiving a message is the same as archiving it, because anyone who can download a message can keep it. And a rule against archiving cannot be enforced against a node whose software has been changed to ignore it.

Stealth's shielded chain already hides what a payment says (Section 4), and a payment note carries a memo. Carrying messages as payments would still fail the fourth requirement. Every shielded transfer permanently adds two leaves to the commitment tree and two nullifiers to the spent set, because an unspent note must stay spendable and a spent note must stay spent. A message sent as a payment could therefore never be deleted, and every message would also consume two of its sender's notes.

9.1 How other systems carry messages

Messaging systems fall into a few families, and each answers part of the problem.

Centralised messengers such as Signal encrypt every message end to end, and Signal adds a post-quantum key encapsulation to its key agreement, so that recorded messages resist a later quantum computer. They charge no fee per message, because one organisation runs the servers and pays for them. The cost is the operator itself: its servers carry every message, an account is registered with a phone number, and whether a delivered message is gone from those servers depends on that operator's software and word.

Messages written into the transactions of a public chain inherit the chain's properties. A Bitcoin transaction can carry a small data output; the data is public, it is kept for good, and the transaction pays a fee. Zcash gives every shielded output an encrypted memo, so a message there is private and travels inside a payment. It pays that payment's fee, its ciphertext stays on the ledger permanently, and its encryption rests on elliptic curves, so a ciphertext recorded today can be read once a large quantum computer exists. Hive needs no transaction fee: an account spends resource credits that come from its staked tokens and regenerate over time. A Hive transfer can carry a memo that the sender encrypts to the recipient, while the transfer itself is public and permanent.

Session carries messages outside any chain. A network of staked nodes stores each message for a limited time in a small group of nodes and delivers it over onion routing, so that no single node knows both origin and destination. An account needs neither a phone number nor a fee per message. Deletion there, as on any network of storage nodes, rests on the nodes discarding what they are asked to discard.

A single operator gives freedom from fees and a place from which messages can be deleted, but it asks to be trusted. A ledger removes the operator, but it charges for every message and keeps every message for good, and the chains above encrypt, where they encrypt at all, with elliptic curves.

9.2 Stealth's approach

In Stealth a message is its own consensus object on the private chain, separate from payments. Four choices define it.

A message pays no fee. What rations messages is a "messaging bond" that a sender locks once, a small proof of work that every message carries, and a fixed cap on the number of messages in a block. The bond is a lock, not a payment: it is never slashed, and it is released in full when its owner unbonds. Sending a hundred messages therefore costs a hundred small proofs of work and no fee, and reading costs nothing. The price of sending is the opportunity cost of the locked bond, chosen because a bond, unlike a fee, limits how many senders an attacker can field without charging honest users for each message.

An example shows the difference. Suppose Alice and Bob exchange two hundred messages in a week. On a chain that charges a fee for every transaction, they pay two hundred fees, and every one of their messages stays on the ledger for good. In Stealth, each of them locks a messaging bond once. Their wallets compute two hundred small proofs of work between them, no fee is paid, and when either of them unbonds, the whole bond comes back. None of their messages is still held by an honest node two weeks after it was sent.

A message leaves no permanent state. It adds no leaf to the commitment tree and no nullifier to the spent set, because it is never spent. It expires at the latest two weeks after it is sent, and after that honest nodes discard it; its only lasting trace is its share of a commitment in its block's header.

A message is encrypted with ML-KEM-768, the post-quantum KEM that encrypts notes, so a message recorded today stays unreadable once a quantum computer exists.

A message is authorised without a signature. A STARK proves that its sender holds a bond, without showing which bond, and the proof is hash-based like every proof of the shielded chain.

Where the messages are held matters as much as how they are built. The private chain is served only to stakers, and the official node software discards message payloads once they expire. The machines that keep message payloads in their block stores are therefore a known set, bound to run that software, not anyone on the internet.

9.3 Sending and reading a message

Suppose Alice holds a messaging bond and wants to send Bob a message. Her wallet encapsulates a fresh key to Bob's encryption public key with ML-KEM-768 and encrypts under it, with authenticated encryption, a payload that carries Alice's authenticated address and the text. This is the construction that delivers notes (Section 4), applied to a message. Every payload is padded to one fixed size, so that a short message and a long one look alike; the same fixed size is the consensus bound that protects nodes against being flooded with large objects.

The wallet then builds the message object. Besides the encrypted payload, it carries a fresh random "replay tag", an expiry, the main-chain height to which the message is bound, a feework computed as for a shielded action (Section 4.7), and a STARK proof. The replay tag is a fresh random number that the wallet draws for each message; the chain keeps the tags of unexpired messages, and so recognises and rejects a message it has already applied (Section 9.4). The proof shows that its creator controls a bond, of the amount required when it was paid, inside a committed set of bonds; the amount is a chain parameter, 1,000 XSS at launch, that a stake-weighted vote of the stakers can move by at most 20% per period, and consensus checks the bond root that the proof refers to against a recognised set of bond roots. A commitment to the payload is bound into the proof's transcript together with the message's other fields and the network identifier, so a proof cannot be moved to a different payload, tag, expiry or network. Messaging uses no user signature anywhere.

Alice's wallet sends the object to its gateway over Tor, as it sends a payment (Section 4.9). The gateway verifies the proof before it passes the message on, the message travels between stakers to the producer of a private block, and the producer includes it. A private block commits to the messages it carries in its header, alongside its actions and burns. A node accepts a message only if all of the following hold:

1. its payload has the fixed message size;
2. its bond proof verifies, against a recognised bond root;
3. its network identifier is the node's own;
4. its expiry lies between the main-chain height to which it is bound and that height plus the maximum lifetime;
5. its replay tag is neither in the guard set of unexpired messages nor repeated within the same block;
6. its feework is valid and recent enough;
7. its block carries no more messages than the fixed per-block cap.

Bob needs no bond to receive or read a message: receiving is free, and the bond unlocks only sending, so Bob needs one to reply. His wallet asks the gateway for all messages in a range of heights, through a read interface that requires no key, and tries to decrypt each payload on his device; the ones that decrypt are his. The chain carries no recipient tag, so neither the gateway nor any node learns which messages Bob's wallet kept: every reader fetches whole ranges, and whether a message is the reader's own is decided only on the reader's device. A message is never spent, so it needs no commitment tree and no proof that it sits in one.

Reading is bounded by the lifetime. Bob's wallet has to fetch a message before it expires, at most two weeks after its sending, and so before honest nodes discard it; a message is not part of a wallet's recoverable state either: restoring a wallet from its seed restores its notes but no messages. Both follow from the purpose of the design, since a message that any node or any seed could bring back would not be deletable.

9.4 Expiry and replay

Every message has an expiry. Consensus bounds it between the main-chain height to which the message is bound and that height plus a maximum lifetime; without the upper bound, a sender could set an expiry so distant that the message never left the network's storage. The lifetime is two weeks, a fixed consensus constant that cannot be changed by governance.

A message must not be applied twice. Each message carries a replay tag, and the chain keeps a guard set of the tags of unexpired messages: a tag is added when its message is applied and evicted once the message's expiry falls below the bound main-chain height. A node rejects a message whose tag repeats within the same block or is already in the guard set. Each block admits at most a fixed number of messages, a tight cap that protects block space for payments, so the guard set is bounded by that cap times the lifetime in blocks and, unlike a nullifier set, never grows without bound. Replay protection is a consensus validity rule that every node evaluates identically; it does not rely on stakers behaving honestly.

Each tag is fresh uniform randomness, never derived from a secret of the sender, so two messages from the same sender cannot be linked by their tags. A message spends no note and publishes no nullifier, so it adds nothing to the sender's note graph either.

9.5 Deletion

The official node software lets its user find their own messages and then discards message payloads. It has no mode that keeps message payloads, and running it is a condition of being a staker. Honest nodes prune a payload after its message expires, and the consensus expiry check means that a pruned message can never be reintroduced. Payloads are pruned only below the private chain's reorganisation and finality floor, never inside the window that a reorganisation can still rewind. Old blocks are validated through the finality-attested header path, never by recomputing the message commitment from payloads, so pruned payloads never have to be kept.

Suppose Alice's message to Bob is bound to main-chain height h and expires two weeks of blocks later, at height e. Bob's wallet fetches and reads it on the third day. Once the bound main-chain height passes e, the message's replay tag leaves the guard set, and each honest node discards the payload as soon as the block that carries it lies below its reorganisation and finality floor. A copy that someone presents again after that is rejected, because its expiry lies below the bound height of any block that could include it.

What remains of a message is its share of the header's commitment to the block's mes-

sages, which finality certified. The fact that some message was sent in a block is therefore permanent, because the header has to commit to what the block carried for the block to be validated at all; the payload, which alone ever held the message's content, is gone from honest storage.

The private chain itself is served only to stakers, and access requires mutual authentication between peers. Authentication has to be mutual, because a one-way challenge can be defeated in real time by a non-staker who relays an honest staker's live response. A node that is not a staker therefore cannot follow the private chain and keep its payloads as they arrive.

Readers are served differently. The interface through which wallets fetch encrypted payloads requires no key and limits each caller's rate. Keeping it keyless is what lets a user without a staker node receive messages at all; in return, anyone can collect the ciphertexts of the message window through it without being identified, and against that collector the design relies on encryption.

9.6 Group chats

A group chat has up to eight members. Suppose Carol, Dave and Erin form one. The members share one symmetric group key, and each group message is exactly one message object with one encrypted body under that key, readable by every member. Carol, who creates the group, delivers the group key to Dave and Erin through ordinary one-to-one messages, encrypted with ML-KEM-768 like any other, and joining a group requires the invitee's consent, so no one is placed in a conversation without accepting it. Each group message is encrypted under a key derived from the group key and a fresh salt, a random value chosen anew for each message, so every message key is unique. Inside the encrypted body, a symmetric MAC (message authentication code) per member attributes each message to its sender.

A group message reveals nothing about the group. Sending a separate copy to each member is rejected, because it would publish correlated objects for every message and leak the group's size and membership to traffic analysis. A group message needs no key encapsulation, since its members already hold the key, so its transport field carries a decoy shaped like a KEM ciphertext, and an on-chain observer cannot tell a group message from a direct message. A two-member group and an eight-member group produce byte-identical sends. Members find group messages by trial decryption alone; no group tag is published, since one would cluster a group's messages and reveal the group's existence and activity. Consensus never sees a group's roster, its size, a group identifier or a recipient tag; the member list exists only on the members' devices.

Removing a member means re-keying. Carol delivers a fresh key to the remaining members only, so a removed member cannot read later messages, while earlier messages stay readable to them. Re-keying is not atomic: until the last remaining member has adopted the new key, messages sent under the old key stay readable to the removed member, who can also forge authorship under it while still holding a bond.

The group key sets the group's boundary of trust. Any member can hand it to an outsider without being detected, so a group is as confidential as its least trustworthy member, and re-keying protects only the messages that follow. Attribution uses a symmetric MAC because messaging uses no user signature: the MAC stops outsiders from forging a sender, but not members, since every member can derive every member's MAC key from the group key. Within a small group whose members chose each other, attribution among members rests on that choice.

9.7 What an observer learns

Table 3 collects what each party can learn about a message.

Messages are not hidden among payments. The bond proof marks a message publicly as a message, so the senders of messages form their own anonymity set, the population of bonded senders, and a sender hides among them. In return, a message spends no notes and publishes one fresh, unlinkable tag, whereas a message carried as a payment would publish real nullifiers and commitments tied to its sender's note graph. The feework of a message, like that of an action, references a recent block and so places the message's creation within a short window.

Party	Learns	Does not learn
Anyone reading the chain or the read interface	that a message was sent in a block; its expiry, replay tag and bound height; the bond root its proof refers to; its fework and the height it references; the padded ciphertext	sender, recipient, text or length; which bond stands behind it; whether it is a direct or a group message; any link between two messages
A gateway or serving node	in addition, when a wallet fetches a range of heights, over Tor	the wallet's IP address; which messages the wallet can decrypt
A group member	the group's messages and the member each is attributed to	other groups; messages addressed to others

Table 3: *What each party learns about a message, and what stays hidden from it.*

9.8 Security and assumptions

The messenger rests on four assumptions.

- **M1. Encryption.** ML-KEM-768 and the authenticated encryption built on it keep payloads confidential, also against a quantum adversary.
- **M2. Proofs.** The STARK is sound and zero knowledge, and the two hashes that bind a message's fields into it are collision resistant: the Rescue-Prime hash that commits to the payload, and the BLAKE3 hash of the proof's transcript, which absorbs that commitment together with the message's other fields.
- **M3. Finality.** Less than one third of the finality committee's weight is Byzantine (Section 7).
- **M4. Honest storage.** Stakers run the official node software, which discards payloads after expiry.

Under M1, confidentiality holds against anyone who does not hold a recipient's key, including a quantum adversary, and it holds wherever a ciphertext is kept: only a participant in a conversation can reveal a message, by disclosing their own decrypted copy. A group's messages rest on the same assumption, because the group key travels in messages encrypted with ML-KEM-768 and a symmetric key loses to a quantum computer only Grover's square-root speed-up.

Under M2, only the holder of a bond can send, and the proof reveals which bond no more than a spend proof reveals which note. A proof cannot be lifted onto another payload, tag, expiry or network. Replay protection and the bound on the guard set are consensus rules that every node evaluates identically. Under M3, a block whose message payloads have been pruned is still validated, through its certified header.

Flooding is bounded without a fee. An attacker who wants to fill the network with messages has to lock a bond before it can send at all, computes a proof of work for every message, and can fill at most the message cap of each block; payments keep the rest of the block, and the guard set stays bounded however long the attack lasts. The attacker's cost grows with every message it sends, while an honest sender's cost is a bond it gets back and a little work per message.

Deletion is weaker, and deliberately stated so: under M4 it is a property of honest nodes. No bond can be slashed for archiving, and a staker running a covert archiving build looks identical on the wire and cannot be detected by the network. Deletion therefore rests on deterrence against identifiable operators, not on detection by the protocol, while confidentiality holds regardless, through encryption.

The same reasoning governs abuse. The protocol has no moderator and can have none, because no one but a participant can read a message.

The messenger thus puts the machinery of the shielded chain, its KEM, its proofs and its staker-only distribution, to a different use: a conversation that pays no fee, hides its participants and its content, and leaves the network's honest storage when it expires.

10 Commit-reveal ("enclave") addresses

A transparent coin's exposure to a quantum adversary has a precise shape (Section 1). While a pay-to-public-key-hash coin sits unspent, only the hash of its public key is on the chain; the full key appears in the spending input. The key becomes public at relay, before and independently of block inclusion, so even a spend that is relayed and never confirmed has exposed its key. From that moment a quantum adversary has a window, lasting until the spend is final, in which to front-run it. And a key that is already on the chain, through a reused address, a pay-to-public-key output or an earlier spend, is permanently forgeable and cannot be protected in place.

Commit-reveal closes the window by separating the moment the destination is fixed from the moment the key is revealed. It defines the spend safety of the transparent chain, anchored in BFT finality. The enclave is a permanent, self-custodied place where the key stays unexposed until it is spent. The shielded chain and the enclave are both quantum-safe, by different routes: the shielded chain by cryptography, the enclave by timing.

10.1 The enclave output

One protocol upgrade, PqCommit, enables both halves of the mechanism: the commit and the protected enclave output type. An enclave output is a standard pay-to-public-key-hash script prefixed by a dedicated vault opcode, so the output script itself marks the coin as protected; like an ordinary output of that kind, it commits only to the hash of its public key. The vault opcode executes as a no-op, so an enclave coin is spent through the ordinary ECDSA signature check. Whether a coin needs commit-reveal is decided only from the resolved coin's own script, never from data the spender supplies and never through pay-to-script-hash wrapping. Consensus requires the commit-reveal path only for coins in the protected type; an ordinary pay-to-public-key-hash coin spends as before, and the upgrade conscripts no existing coin. An enclave address is a bech32m string that begins with `xst`, and an enclave coin can be received directly from the shielded chain, in one step, by unshielding to an enclave address. Protection is therefore opt-in: a coin never moved into the protected type stays exposed once a quantum computer exists, and a coin whose key is already on the chain needs a one-time sweep to a fresh address, completed before then, since a sweep still in flight at that point can be front-run. Only single-key pay-to-public-key-hash coins are protected; the enclave type has no multisignature or pay-to-script-hash form. Moving an enclave coin into the shielded chain takes two hops, a commit-reveal spend to a fresh plain address and then a normal shield.

10.2 Commit

To spend, the holder first publishes a "commitment" C:

$$C = \mathrm{SHA256d}\big(\text{network tag} \parallel \text{scheme version} \parallel \text{public key} \parallel \text{outpoint} \parallel \text{destination-set hash}\big),$$

where SHA256d is SHA-256 (the Secure Hash Algorithm with a 256-bit output) applied twice. The network tag is the double SHA-256 of a label for each network, so a commitment made on one network never matches a reveal on another, and the scheme version ensures that a commitment under one binding scheme never matches a reveal under another. The destination-set hash covers every value output of the spend in transaction order, change included, each with its amount and its length-prefixed script. The only output it excludes is a trailing feework proof, identified by its type rather than its position, and that output must carry zero value, so no value escapes the binding.

The commitment binds where the coins go, not the transaction's identity. At the reveal, the holder may redo the feework, add inputs or re-sign and still match, while every value output stays fixed.

A commit is a transaction of a fixed shape, and the commit itself is feeless: an expiring, memory-hard feework replaces any fee. The rules are:

1. a commit is exactly one transaction with one phantom input, an empty signature script and exactly one output, its feework proof;
2. that output carries zero value, so a commit moves no coins and spends no funds;

3. the commit carries no separate commitment output: its phantom input is the domain-separated hash of the commitment, which alone binds the commitment and gives each commitment a unique input, so two commits never collide; it is not resolved against the coin set, so publishing a commitment spends no coin and exposes no key;
4. a block is invalid if any commit is outside this shape;
5. a block may carry at most 256 commitments;
6. the commit's feework must reference a block no more than a small, fixed number of blocks below the tip, so a commit expires shortly after it is ground;
7. the commit's feework must meet a required memory cost, set by how full the commitment cap was over a buried window of blocks, so every node computes the same value and no committer can choose it.

On connecting a block, each node records every commit it carries, by the hash of its commitment, together with its inclusion height; no outpoint is stored. A commitment is dropped from this index once the tip has passed its inclusion height by a burial offset of one finality epoch, 512 blocks, plus a reveal window of 720 blocks, about an hour, whether or not it was spent. In a reorganisation, the index drops commitments included above the fork point, so it always reflects the active chain.

10.3 Reveal

The "reveal" is the ordinary ECDSA spend of the enclave coin, accepted only if a matching earlier commitment exists. The node takes the public key from the reveal's signature script and recomputes the commitment from that key, the input's outpoint and the reveal's actual outputs. The reveal is valid only if the hash of the recomputed commitment is in the index and its inclusion block is at or below the finalized height; otherwise the spend is rejected. The rule runs for every input that spends an enclave coin, so a spend of several enclave coins needs a finalized commitment for each, all over the spend's one set of outputs, and mempool admission applies the same rule as block connection, so a reveal that relays can also connect. qPoS registry keys are outside commit-reveal: they are on the chain by construction and authorize changes of state through operations with a single input, which must not be an enclave coin, so the rule for inputs never applies to them.

Continuing Alice's example from Section 6: her commitment fixes her key, her 40 XST coin, 30 XST to Bob and 10 XST to herself. When she reveals, an adversary who derives her private key can sign a spend, but any change to a recipient, an amount or the change yields a different C, which has no finalized match.

An enclave address can be funded many times. Consensus does not stop it being funded again, but its first spend publishes the public key, and from then on every coin at that address, whether it arrived before or after that spend, is exposed.

10.4 Security and assumptions

C is a hash, so the public key inside it stays hidden, by the hash's preimage resistance, until the owner reveals it. Each field of the commitment is there for a reason. Binding the public key defeats front-running of the commit itself: an attacker cannot form a matching commitment before the key is public. Binding the outpoint pins the coin a commitment authorizes, so a foreign commitment or a substituted input cannot ride on another coin's commitment. And binding the destinations stops redirection. A commitment to the public key alone would not suffice, because an attacker could then ride the victim's own commitment with a redirecting spend.

Burial depth is not a security parameter: spend safety anchors to BFT finality, and a reorganisation that forks below the finalized height is rejected at any chain weight. The burial offset is the fixed offset at which commitments are pruned, and it stays a constant rather than the live finality lag, or a finality stall would let the index grow without bound. Pruning by time needs no test of whether a coin is unspent, so the index never stores the outpoint and never reveals which coin a commitment protects before the reveal.

A protected coin is safely spent once its reveal block is final. Replaying a reveal fails,

because the spent coin has left the coin set: a commitment only gates a spend and never authorizes one. A reveal that never confirms loses nothing; the coin never moved, and the wallet issues a new commitment for it, with a new change key derived from the wallet's seed or, when the reveal leaves no change, with new destinations the owner chooses. An enclave address from which no coin has yet been spent is quantum-safe because inverting its public-key hash is quantum-hard; Grover's algorithm gives only a square-root speed-up. Commits are rate-limited not by money but by the work and the expiry that every commit carries.

Two residual risks remain within the finality trust boundary. The front-running window that is left is the finality lag between the reveal's confirmation and its finalization, and a party that stalls finality holds it open. A finality majority that censors can suppress a reveal after its key is exposed; this is accepted as the same trust boundary as any high-value settlement, and finalization is the mitigation. No block space is reserved for reveals, because an unconditional reservation would be dead space whenever reveals did not fill it, so the protocol gives no hard guarantee that a reveal confirms inside the window. Starving a reveal gains an attacker nothing, because funds are never stealable through it; the attack only lengthens the window in which a key is exposed. An attacker who grinds commits to fill the per-block commitment cap can delay a commit, but coins in unexposed enclave addresses are never at risk.

11 The trustless light client

A wallet that does not run a full node learns the chain from a server. Unless it can check what the server reports, it must trust that server on whether a payment is in a block and whether that block is final. Checking it the obvious way is full-node work: the finality committee changes through transactions and through derived changes such as disabling inactive members and recomputing weights, and following those changes means processing full blocks and applying the committee rules.

The desktop and the mobile app use the same trustless light client, and it takes nothing from the node that serves it on trust, including which committee is current. It trusts exactly two things: a checkpoint built into the app, and the assumption that at least two thirds of the committee's weight is honest, which is the assumption a full node relies on. Instead of re-deriving the committee rules, it verifies a chain of committee signatures.

11.1 Following the committee

The committee is committed at every epoch boundary (Section 7): a boundary block must carry exactly one committee root, or it is invalid, and every full node recomputes that root from its own state and rejects a boundary block whose root differs. The committed committee is the result of every change in the epoch, so derived changes never reach the client as individual events, and the committee's weights are frozen for the epoch, so a certificate is checked against fixed weights. The block that carries a new committee root is certified by the current, already trusted committee, so each committee is signed into place by the committee before it.

The client starts from the checkpoint built into the app, which fixes a committee that the client trusts. Each epoch's boundary block commits the next committee and is certified by the current one. The client checks each ML-DSA-65 signature in that certificate against the key that its trusted committee fixes for that signer, sums the signers' weight, and requires more than two thirds of that committee's total weight. If the certificate checks out, the client adopts the next committee as its trusted committee, and the walk repeats for the following epoch. It adopts committee roots only at or below the finalized frontier, so a reorganisation above the frontier cannot change an adopted root.

11.2 Verifying a payment

Suppose Frank's phone wants to know whether a payment to him is settled. The serving node supplies the header of the block that contains it, the block's finality certificate and a Merkle proof that the transaction is in the block. The client checks the certificate against the trusted committee root of the epoch of the height before the block, and it verifies the inclusion proof against the transaction root, the Merkle root of the block's transactions that every block header carries. The

phone has then established by itself, trusting no server, that the transaction is in a block that more than two thirds of the committee's weight finalized.

11.3 Security and assumptions

Under the assumption that at least two thirds of the weight is honest, a committee transition attested by more than two thirds of the weight the client already trusts is correct, including every derived change, because an honest supermajority computed it by the real rules. A false committee root would need a Byzantine coalition holding more than a third of the weight to sign it, which is slashable equivocation, so the chain's slashing backs the light client too. Finality votes and certificates, including those the client checks, are signed with ML-DSA-65, so the client's guarantee holds against a quantum adversary as well. The serving node is a pipe for headers, certificates and Merkle proofs; if it lies, the client's own verification rejects the data. The one further assumption is that the checkpoint built into the app was genuine when the app was installed. The serving node is trusted for liveness only: a node that withholds data cannot forge anything, but it can fail to advance the client, which then turns to another node. A client that stays offline for longer than the horizon over which the committee rotates may need a fresh checkpoint to re-anchor safely.

12 StealthPay and x402 agentic payments

A software agent can buy resources one request at a time: a page of data, an answer from a model, a minute of computation. The price of one request is small, often a fraction of a cent, and an agent may make thousands of such purchases a day from sellers it has never dealt with. Five facts make paying this way hard. An agent that buys this way holds no account with each seller, so each payment has to carry its own authorization and be checkable by the seller without prior trust. A fee on every transaction is a floor under the smallest payment worth making: where the fee approaches the price, paying per request is uneconomic, and where it exceeds the price, it is pointless. A payment recorded in the clear shows payer, payee and amount to everyone, so an agent that pays per request publishes which services it uses, how often and for how much, unless it can pay privately. A signed payment is data that can be copied, so a seller must bind each payment to a single request, or the same payment can be presented again. And a program that holds spending power can spend all of it when it malfunctions or is compromised, so limits on its spending have to be enforced outside it.

12.1 How other systems pay per request

HTTP (the Hypertext Transfer Protocol) has long reserved a status code for such payments: 402, "Payment Required". x402 is a payment protocol built on it. A server answers an unpaid request with status 402 and a description of the payment it accepts; the client pays and repeats the request with the payment attached; and a "facilitator" verifies and settles the payment for the server. x402 is not tied to one chain. In its scheme for Ethereum-style chains, the payer signs an authorization to transfer a token, limited by a validity window and by a nonce that stops it being used twice, and the facilitator submits it to the chain and pays the network fee. The payer need not hold the chain's own currency, but the fee remains: the facilitator pays it, so it has to hold that currency and bear the cost.

L402 pairs the same status code with the Lightning Network. The server's challenge carries a Lightning invoice together with a token, and the client presents the token with the invoice's preimage as proof of payment. Lightning moves payments off the chain into channels between parties, where they are fast and cheap. Opening and closing a channel are on-chain transactions that pay a fee, a routed payment pays routing fees, and a payment needs enough liquidity along its route.

A few chains carry transactions without a fee. Nano replaces the fee with a small proof of work in every block of an account's chain. Hive lets an account transact without a fee by spending resource credits that come from its staked tokens and regenerate over time. The transactions of both are public.

12.2 Payments without a fee

Stealth's transparent chain belongs to this last group: its ordinary transactions need no fee and carry a feework instead (Section 14). Without a fee, the smallest payment is set by the smallest output the chain accepts, and at and after the finality fork that is one atomic unit, a millionth of an XST (Section 15). An agent can pay a thousandth of an XST for a request, and the seller receives exactly that amount; nothing is deducted on the way.

The absence of a fee also changes who has to hold what. A feeless transaction is complete once its payer has signed it: no third party has to submit it and pay a fee on the payer's behalf. The signed transaction is therefore itself the payment's authorization, and the facilitator that checks and broadcasts it needs no funds of its own, and on the transparent rail no spending key either.

The feework does the work that a validity window does elsewhere. It is computed over a recent block, and a feeless transaction whose block lies too far below the tip is invalid, so every feeless payment expires by a rule of the chain, not by a clock that the seller has to trust. A fresh receiving address for every challenge binds each payment to one request: a payment pays only the address it names, and each address is issued once.

StealthPay builds on these properties with version 2 of x402. Payments run on two rails: a transparent rail on the XST chain, feeless, and a shielded rail in the XSS pool, on which the chain learns nothing about a payment's payer, payee or amount. Three parties take part. The "agent" is the program that pays. The "facilitator" issues payment challenges on behalf of a seller and verifies and settles the payments made against them. The "operator" owns the agent's funds and enforces its spending limits through a "policy engine".

12.3 Challenge and payment

Suppose an agent requests a resource priced at 0.01 XST. The seller asks its facilitator for a challenge and answers the request with status 402 and that challenge. The challenge names the resource and a single offer: the price, the rail on which it is to be paid, a receiving address that the facilitator issued for this challenge alone, and a timeout. An offer on the transparent rail is priced in XST, and one on the shielded rail in XSS. The facilitator keeps every timeout shorter than the feework window, so a challenge never outlives the payment made against it, and it names a recent block to which the payer may anchor its feework. The agent answers with a payment payload that names the resource, the one offer it accepted and the payment itself.

On the transparent rail, the payment is one fully signed feeless transaction that pays the price to the receiving address; no separate authorization is sent. Its feework is computed over the hash of an anchor block and over the transaction with its feework output removed and its input scripts blanked, so the feework can be computed before the transaction is signed. A payer may therefore have the feework computed by another party, which receives the unsigned transaction, its anchor and its memory cost, and no key. Because the public key that spends a pay-to-public-key-hash coin appears only in its signed input script, the party that computes the work never sees it; for a coin in an enclave address, whose public key stays unknown until its reveal, the key stays hidden from that party as well. A facilitator can offer this service to the payers of its sellers, which suits an agent that runs where computation is scarce; the party that computes the work learns the shape of the transaction, which the facilitator learns at verification anyway.

On the shielded rail, the payer spends two of its notes in one shielded action that creates a note for the receiving address of the challenge and a change note back to itself, and it submits the action to the network itself. The payment it presents is the opening of the note that paid: its commitment, the recipient key, the value and the note's two randomness values, in the form of a payment disclosure (Section 4.6).

12.4 Verification and settlement

The facilitator finds the challenge by the receiving address named in the payment and verifies against its own stored offer, never the payer's copy; offer terms posted alongside the payment must equal the stored ones exactly. A challenge expires once its timeout has passed since it was issued. A transparent

payment is accepted only if all of the following hold:

1. it contains exactly one output that pays the challenge's receiving address exactly the price;
2. every input exists unspent;
3. every input signature verifies under the script rules that apply to the next block;
4. its feework verifies against its anchor block at the hardness the chain requires;
5. its feework anchor does not lie within a safety margin of the point at which feework expires.

Rule 5 leaves a payment accepted now a margin of blocks in which to reach a block before it expires. Verification only reads the chain. Settlement re-runs the full verification against the facilitator's current view of the chain before the payment is broadcast, because the chain and the pool of pending transactions can change between the two steps.

The seller chooses, per challenge, when a transparent payment counts as settled. In the default, confirmed mode, settlement succeeds once the payment is in a block and fails if the challenge's time runs out first. In mempool mode it succeeds as soon as the node accepts the payment into its pool of pending transactions, and an automatic mode settles below an amount set on the facilitator from the pool and above it from a block. The settlement receipt names the network and the settling transaction, with its status and, once the payment is in a block, that block's height. Confirmed settlement waits for a block, not for finality: a request is answered as soon as the payment is in a block, and what a seller risks on a block that is later reorganised away is the price of one request.

On the shielded rail, the facilitator broadcasts nothing; settlement waits for the paying note to appear. The facilitator checks that the disclosed value equals the price and that the disclosed recipient key is the key of the challenge's receiving address, recomputes the note commitment from the opening, and checks its Merkle path to the pool's root. No zero-knowledge proof is verified on this path: the action's own proof was checked when the network admitted it, and the opening shows what the note contains. Settlement waits until the paying note is in the commitment tree under the node's latest recorded root, within the challenge's time, and the receipt identifies the payment by its note commitment.

12.5 Spending limits

An agent run under StealthPay holds no signing key. It holds a credential with which it asks its operator's policy engine to pay, and the engine signs only a payment that passes the agent's policy. Each agent pays from the sub-account of the operator's wallet that the operator assigns to it, so that sub-account's balance bounds what the agent can spend at all. Within it, the operator can:

1. suspend an agent, whose payments are then refused;
2. restrict an agent to a list of resource hosts named in its payment requests;
3. restrict an agent to a list of receiving addresses;
4. limit an agent to a number of settled payments per hour;
5. cap an agent's daily spending and its total spending.

The caps count payments still in flight as well as settled ones: the engine reserves each payment against the caps under the same lock with which it checks them, so two concurrent payments cannot both pass a cap that only one of them fits under. The engine also refuses to build a payment whose amount differs from the amount its policy approved. And an agent that retries a request does not pay twice: each payment request carries a key, given by the agent or derived from the request, a retry with the same key returns the payment already built, and a concurrent duplicate is refused. The operator watches its agents in a console that contains no wallet and holds no spending key, so it cannot make a payment.

12.6 Security and assumptions

Keys are placed so that each party can do only its own part. On the transparent rail, the facilitator derives its receiving addresses from the seller's extended public key, so it holds no key that can spend the funds it receives. The seller still trusts the facilitator in two ways: to issue only addresses derived from that key,

never addresses of its own that would route payments to itself, and, against a quantum computer, not to derive from the extended public key the private keys of every address under it. On the shielded rail it holds spend authority over its receiving addresses, because a shielded address key is the hash of a nullifier key (Section 4) and cannot be derived from public material; a shielded facilitator is therefore trusted with the funds it receives as a wallet is. The agent holds no key at all.

A settled challenge cannot be paid again: verification of any further payment to its receiving address fails, and presenting a payment again returns a receipt that names the originally settled transaction, marked as a replay, and settles nothing a second time. Only one settlement per challenge runs at a time, because settlement moves the challenge from open to settling atomically and refuses a second attempt while one is in flight. Expiry is enforced by the chain itself, since a feeless transaction whose feework anchor is too far below the tip is invalid. Replay protection assumes that no receiving address is issued twice, for which the facilitator records, across restarts, how many addresses it has issued, and that each challenge record stays with one facilitator.

What the shielded rail shows follows from Section 4. The chain sees a shielded action and learns neither payer, payee nor amount. The facilitator is shown the paying note's opening, its value, recipient key and randomness, all of which concern its own receiving address; the disclosure carries nothing about the payer's inputs or other notes. On the transparent rail, a payment is as public as any transparent payment.

Spending limits are enforced by the operator's policy engine, which holds only each agent's sub-account keys, never the operator's recovery phrase; the chain does not enforce them. The limits are as strong as the engine and the machine it runs on, and it runs in the operator's own infrastructure rather than at a third party.

A transparent payment always carries a feework, because the feework is what gives it an expiry that the chain enforces. Under heavy demand the required work rises with the chain's demand floor, and a payer who cannot spare the computation can hand it to another party without handing over a key.

Together these give an agent a way to pay per request that the chain itself makes cheap. No fee sets a floor under the price, no facilitator has to pay a fee on the payer's behalf, a payment's expiry and its binding to one request are enforced by the chain and by single-use addresses, and on the shielded rail the payment reveals nothing to the chain about who paid whom or how much.

Part VI
The Chain

13 Pruning

A chain's history grows with every block, and a node that keeps all of it needs ever more storage. Every private block carries the proofs of its shielded actions, and once a block's effects are in every node's state, its proofs are no longer needed to know that state. Stealth therefore prunes, but only the private chain. The main chain is not pruned; it is the record from which every transparent coin descends. Nodes prune by default.

13.1 What a node keeps

A pruning node keeps private blocks only for a recent window: its "retention floor" is the tip minus 1,024 blocks, and private blocks below the floor are deleted from its block store. Each time the node adopts a new private block, it advances the floor and prunes below it, and the floor only ever rises. What the node deletes is only the block store. Every node keeps the full shielded nullifier set and the full commitment tree in its state, whatever it prunes, and the anchor window lives in that state too. Dropping blocks below the floor therefore removes no anchor and no nullifier: every note can still be spent, and no spent note can be spent again. Pruning thus also removes the private chain's transaction history from every pruning node: below the floor it keeps no record of which actions took place, only their effects in the nullifier set and the commitment tree, which are the private ledger's state.

A node may opt out of pruning and keep the full private-chain history as an "archive" node. Archive nodes are where a brand-new node obtains the early private chain, since pruned peers no longer hold those blocks.

13.2 Serving a pruned chain

A node says what it can serve. It advertises archive service when it can serve history from genesis, and pruned service when it serves only a bounded range. Its private-block announcements carry, alongside its tip, a lowest height one above its retention floor; it still holds and serves the block at the floor itself, so it serves one height more than it advertises. A node answers a peer's announcement with a request only if the peer's advertised range covers the height it needs (Section 5); separately, it asks every peer for its next height at intervals, whatever range the peer advertises. A request below a node's floor is answered with an explicit not-found that names the missing height and the node's servable range, never with silence, and after a not-found the requesting node stops re-requesting that stalled range from that peer in response to its announcements.

13.3 Security and assumptions

Pruning never removes a body that a reorganisation could still need. The retention depth is at least the deeper of the private chain's rewind depth and the main chain's reorganisation depth, so no in-place reorganisation reaches below the floor. It is also at least the finality window's minimum of 1,024 blocks.

14 Feework hardening and block space

Stealth has carried feeless transactions on its main chain since 2021, when the feeless rule switched on at block 3,702,300. The chain is fee-optional: a sender may pay a small money fee, or send without one and attach a proof of work in its place. What keeps feeless sending open without opening the chain to spam is that proof of work, the feework: it costs its sender real computation, and it expires. This section describes that mechanism first, and then the hardening that the new protocol adds to it.

14.1 Feework

A transaction that pays at least the minimum money fee, one cent (0.01 XST), needs nothing more. A transaction that pays less must carry a valid feework, or every node rejects it. Feeless therefore means free of a money fee, not free of cost: the cost is computation, done once before the transaction is sent, by the sender or, on StealthPay's x402 rail, by a facilitator the sender asks.

The fework travels inside the transaction, as its last output. That output carries three numbers: the height of a recent main-chain block, the amount of memory with which the work was computed, and the work itself, a number that the sender searched for. In a transaction that pays less than the minimum fee, a fework output in any other position makes the transaction invalid, and so does a second one.

To find the work, the sender's wallet takes the hash of the main-chain block at the chosen height, followed by the transaction with its fework output removed and the signature scripts of its inputs left empty, and hashes both with Argon2d, salted with a candidate work number. The candidate is valid if the hash is no greater than a fixed limit. Most candidates fail, so the wallet tries one after another until one succeeds. Because the transaction's contents enter the hash, work found for one transaction is worthless for any other: every transaction needs work of its own. Because the signature scripts are left out, the work can be computed before the transaction is signed, and each signature then covers the finished work through the transaction hash.

The limit is fixed. On average about 9,362 candidates must be tried before one meets it, while a node checks a fework by computing a single hash. Producing the work is thus thousands of times harder than checking it, and every node can check every feeless transaction at little cost.

Argon2d is memory-hard: each evaluation must fill and read a region of memory of the declared size, so every try costs memory as well as computation. The smallest fework uses 256 kibibytes per evaluation; a larger transaction needs 256 kibibytes more for every full thousand bytes. No fework may declare more than 4,608 kibibytes, and no single feeless transaction may be larger than a fifth of a block. A feeless transaction may spend only coins that are already confirmed in a block.

14.2 Expiry

The fework is bound to a moment as well as to a transaction. The block it names may lie no more than 24 blocks below the tip that validates it, and it may not lie above that tip. At one block every five seconds, 24 blocks are two minutes. The C++ node and the Rust node check the same depth.

Once the named block falls deeper, the work is void. The transaction can no longer enter a block, and after each new block every node removes it from its pool of waiting transactions, without computing any hash again. A sender whose transaction has expired sends it again, with fresh work or with the money fee.

Expiry is what makes the mechanism hard to beat. A block's hash cannot exist before the block, so the reference fixes the earliest moment at which the work can have been done, and every node agrees, block by block, which transactions have expired. Work therefore cannot be computed in advance and stored for a later flood. A spammer who wants to keep the pools of the network full must replace their entire contents with freshly computed, valid transactions every two minutes, for as long as the flood lasts, while an honest sender computes the work once per payment. When the spammer stops, the last of the spam expires about two minutes later, and the pools are as they were.

The private chain's actions carry fework of the same kind: Argon2d over the action and the hash of a recent main-chain block, at a fixed 256 kibibytes and the same limit, void once that block lies more than 24 main-chain blocks below the height to which the action's private block is bound. Its memory does not grow with size, and that chain has no money fee to fall back on.

14.3 What hardening changes

The mechanism as it stands has a gap that the new protocol closes. Consensus checks each fework only against the memory that its transaction's size requires; the extra memory that a fuller block would call for is chosen by the sender's wallet from the fullness of recent blocks, not required by the rules that every node enforces, and the difficulty limit never moves. A spammer therefore pays no more at the height of congestion than at its quietest. And while no single feeless transaction may exceed a fifth of a block, nothing bounds the feeless bytes that a block carries in total or keeps space free for transactions that pay a fee, so one producer could pack a block with

feeless bytes that it made itself and distort any signal read from fullness.

Fework hardening makes the price of feeless space follow demand, by consensus. Every node computes a minimum fework difficulty for transparent feeless transactions from how full recent blocks were with feeless transactions, measured over a window of buried blocks whose newest lies 25 below the block being connected. Demand lowers the limit that a hash must meet, so each transaction needs more tries, while the memory of each try stays pinned. The floor rises as demand rises and falls again as it eases, and its ceiling is set aggressively. A consensus byte cap bounds the feeless share of every block, and the rest is reserved for transactions that pay the money fee. Both come in at the finality fork.

The ramp is on difficulty, not on memory, for the sake of the nodes that check it. A node verifies a fework by computing one hash at the declared memory, so raising the memory of each try would make every check more expensive as well. Lowering the limit leaves that check at one hash however high demand climbs, so the floor can span a far wider range without adding to any node's burden: the spammer's cost grows, the verifier's does not.

The demand signal is read over a window, not from the last block alone. A rule keyed to the single previous block could be played: fill one block cheaply, sit out the expensive block after it, and return when the price has fallen again. Averaging the feeless fill of several buried blocks blunts that game and lets the floor rise and fall gradually.

The design is already in the protocol for one kind of feeless traffic. Commit carriers, the transactions that create commitments for commit-reveal addresses, need fework whose memory rises with the number of commits in a buried window of 24 blocks whose newest lies 25 below the block being connected; the requirement is therefore the same for every node, independent of the order of transactions within a block, and out of reach of the sender's choice of reference block. The hardening carries that design to every transparent feeless transaction, StealthPay's transparent payments among them.

The private chain is left as it is. Its fework keeps a fixed limit, because that chain has no money fee to fall back on: a demand floor there could price honest actions out with no way around it.

14.4 The demand floor

The window's offset is chosen for a reason. Because it lies one block beyond the expiry depth, every height at which a transaction can still be included keys to blocks that were already known when its fework was ground. A demand spike after a transaction is admitted can therefore never raise that transaction's requirement at any height where it can still be included.

Suppose Alice's wallet grinds fework for a payment while the network is quiet, and a spammer then fills the feeless space of the next blocks. Alice's payment is still judged against the window that was known when she ground it, so it stays valid. The spammer's own later transactions are keyed to the full window and need more tries each. If demand stays high long enough that Alice cannot meet the requirement in reasonable time, she attaches the minimum money fee instead, and her transaction needs no fework at all.

The floor is applied identically at every enforcement point: block validation, block building, mempool admission and every producer that grinds fework. Producers therefore never build a block that the network rejects for its fework.

14.5 Block space

Block size is a consensus rule. A block is invalid if its serialized size exceeds the block-size limit, or if it has more transactions than that limit allows; the limit is a consensus parameter that depends on block height, and the per-block cap on signature operations is derived from it. Within that limit, a consensus byte cap bounds feeless transactions to 90% of each block's space and reserves the other 10% for transactions that pay a money fee: when the feeless lane is jammed during an attack, that reserve is the way out. The bound is on bytes rather than on a count of transactions, because a byte bound limits feeless bytes exactly, whatever the size of each transaction.

14.6 Security and assumptions

The required difficulty is a pure function of blocks already connected, with no wall clock and no dependence on the order of transactions within a block, so every node computes the same requirement. Manipulating the floor requires a reorganisation at least 25 blocks deep, one block more than the feework expiry. Pinning memory keeps the cost of verification constant regardless of demand, because each verification is one Argon2d evaluation at the declared memory cost. The consensus cap on each block's feeless bytes also bounds how far any one block can move the demand signal. Pinning memory has a price: under high demand the work becomes a pure count of hashes; it is paid so that verification stays equally cheap for every node at every level of demand.

15 Microtx: the minimum output

A chain that rejects outputs below a minimum value cannot carry payments smaller than that minimum. In the C++ node, the minimum standard output equals the minimum transaction fee, which is one cent, 0.01 XST. A payment of a fraction of a cent cannot be made.

The new protocol lowers the minimum output from one cent to one atomic unit, the smallest amount the chain can represent, which makes microtransactions ("microtx") possible. One XST, like one XSS, is 1,000,000 atomic units, as before, so the new floor is 0.000001 XST, and no amount finer than one atomic unit can be written. The change switches on with the finality fork:

1. below the finality fork, the standard-output floor stays at one cent;
2. at and after the finality fork, the floor is one atomic unit;
3. the floor applies to the spendable output types (pay-to-public-key, pay-to-public-key-hash, pay-to-script-hash and bare multisignature) and to protected enclave outputs;
4. outputs that claim qPoS rewards are subject to the same height-gated floor;
5. protocol marker outputs of zero value are exempt from the floor, but may not be negative.

Consensus rejects a spendable payment output below the floor in force at its height.

What stays unchanged matters as much. The minimum money fee stays at one cent, so lowering the output floor does not lower the fee floor. A transaction that pays no fee is feeless and carries feework (Section 14).

16 The +100 StealthNodes

Every qPoS staker is a StealthNode. A StealthNode produces the main-chain block of its five-second slot and the private block bound to it, checks the blocks of every other StealthNode, and, once enabled, sits on the finality committee with its qPoS weight. The StealthNodes are therefore the parties that make, check and finalize the chain.

The protocol has offered 100 StealthNodes, and all 100 are sold. A StealthNode is bought from the chain itself, by a purchase transaction whose price is burned; the price is set for each block from the number of qualified StealthNodes and the money supply, so it rises as StealthNodes are sold. An owner can sell a StealthNode at any time, by transferring its ownership to the buyer. But a holder who wants to run a node, and finds no owner willing to sell, has no other way in.

The first reason for more StealthNodes is that market. The expansion offers 100 more, so that whether one can buy a StealthNode from the chain is answered yes again, for as long as any remain. They are sold first come, first served, at the price that the protocol sets; a purchase that does not get through is made again at the next price. The added StealthNodes continue the price curve of the first 100, so at the same money supply each costs more than any of those.

The second reason is the bound that finality needs. A certificate that proves a quorum by listing its signers' signatures, as the finality certificate does, grows with the committee, so a chain that finalizes blocks with such certificates needs an upper bound on its committee to keep them bounded.

The +100 StealthNodes answer both. A hard fork, with its own place in the proto-

col's ordered fork schedule, raises the number of StealthNodes from 100 to a ceiling of 200, and consensus enforces that ceiling from the fork on. Each StealthNode comes with its own NFT (non-fungible token); the 100 added StealthNodes take theirs from a second batch of 100.

16.1 Buying and enabling a staker

A staker is created by a purchase transaction, which registers the new staker under the next sequential staker identifier. The price of a staker is computed from the number of qualified stakers and the money supply, as the qPoS paper describes. A purchase must pay at least the current price and may pay up to twice the current price, so that it still succeeds if the price rises while it waits. The price is burned: a purchase transaction's inputs must cover its outputs plus the price, and no output receives the price. At or after the fork, a purchase made once 200 stakers are registered registers no staker keys.

A newly purchased staker starts disabled and produces no blocks until it is enabled, by an operation signed by its owner, manager or controller key. The finality committee is the set of enabled stakers, each with its qPoS weight, so an enabled new staker joins it; because the committee is frozen at each epoch boundary, the new staker enters the committee at the next boundary. Each enabled staker adds one five-second slot to the qPoS round, so the more stakers are enabled, the less often each of them produces a block. A staker registered later has lower seniority, and so no higher weight at equal production, than older stakers, and each new registration raises the seniority of every existing staker.

16.2 Security and assumptions

Enabling a staker is accepted only when it is signed by a key registered to that staker. A purchase that registered no keys can therefore never be enabled, and a staker that is never enabled neither produces blocks nor joins the finality committee; this is what makes the ceiling hold. The ceiling limits only new registrations; stakers registered before the fork stay registered. The ceiling counts every staker ever registered, and staker identifiers are never reused, so once 200 stakers exist, a staker lost to disqualification is not replaced; this keeps the bound on finality certificates fixed.

The ceiling also bounds finality. Stakers 101–200 come in the fork after the finality fork, and there are never more than 200, so every finality committee lies within the 200-staker ceiling. The maximum number of signatures in a finality certificate is defined as the staker ceiling, 200, and a certificate with more signatures is rejected when it is decoded. At an epoch boundary, a committee of more than 200 members is not frozen, and finality halts when it reaches that epoch rather than tally against an unbounded committee.

Part VII
Closing Remarks

17 Closing remarks

This paper began with two problems: a public ledger exposes every payment, and a quantum computer breaks the signatures and the encryption on which chains rely. The parts of the protocol answer them together, and this section shows how they compose.

Finality is the joint that holds the parts together. BFT finality (Section 7) turns qPoS settlement into a guarantee, and the other parts build on that guarantee rather than on burial depth. Commit-reveal gives transparent spends their safety against a quantum adversary, anchored in finality (Section 10). Custody of bridged value requires finality: the chain releases locked XST only against a root that the committee has ratified (Section 8). All shielded settlement is paced to the finalized frontier, and the light client follows the chain by checking the same committee signatures (Section 11). Beyond the cryptography, one assumption carries the security of all of them: that less than one third of the committee's weight is Byzantine, the same trust already placed in every block. The light client adds only that the checkpoint built into the app is genuine.

The cryptography is chosen so that each protected route keeps its guarantee when a quantum computer exists. Shielded spend authority rests on hash preimages proved by a hash-based STARK, which needs no trusted setup; shielded confidentiality rests on ML-KEM-768; blocks and finality votes are signed with ML-DSA-65 (Section 6). The private messenger inherits the same encryption and the same hash-based authorization (Section 9).

The holder chooses the route, and each route is complete in itself. The shielded chain is the quantum-safe option that needs no further action: a payment there is private, final and feeless, and its notes stay confidential and unforgeable against a quantum adversary. Ordinary XST stays feeless and transparent, spent with ECDSA as before, and a user priced out of the feeless lane can pay a small fee (Section 14). And an enclave is the opt-in for holders who want transparent coins theft-proof against a quantum adversary. The limits of each route are stated where the route is described: what the gateway learns (Section 4), what the boundary of the shielded pool shows, and which coins cannot be protected in place.

Beneath all of this runs one node, and at its centre one thread. The Rust node validates the existing chain exactly as before and adds every new consensus rule at a fork height, several rules sharing one fork height. It runs the main chain and the private chain on one consensus thread with one clock: every block, every produced slot and every admitted transaction is a command to that thread, taken in the order in which it arrives, and the private chain steps at the main-chain blocks it is bound to. The applications of Part V, from the messenger to StealthPay, use the same chain.

Disclaimer

This paper describes a protocol and is not investment advice. Anyone who uses the Stealth protocol or its software does so at their own risk. Nothing in this paper is a promise.

This paper is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0): `https://creativecommons.org/licenses/by/4.0/`